

Competitive Programming

Section 61 Editorial: ICPC Contest Readiness

Competitive Programming Course

2026-07-18

Table of contents

1	How to read this editorial	9
2	Problem A: Scoreboard Replay	10
2.1	Restatement	10
2.2	Important constraints and traps	10
2.3	Observation	10
2.4	Sample walk-through	10
2.5	Algorithm	10
2.6	Correctness proof	11
2.7	Complexity	11
2.8	C++ implementation notes	11
2.9	Python implementation notes	11
2.10	Full C++ solution	12
2.11	Full Python solution	12
2.12	Common wrong answers	13
2.13	Tests that matter	13
3	Problem B: Time-Boxed Schedule	14
3.1	Restatement	14
3.2	Important constraints and traps	14
3.3	Observation	14
3.4	Sample walk-through	14
3.5	Algorithm	14
3.6	Correctness proof	15
3.7	Complexity	15
3.8	C++ implementation notes	15
3.9	Python implementation notes	15
3.10	Full C++ solution	15
3.11	Full Python solution	16
3.12	Common wrong answers	16
3.13	Tests that matter	17
4	Problem C: First-Round Assignment	18
4.1	Restatement	18
4.2	Important constraints and traps	18
4.3	Observation	18
4.4	Sample walk-through	18
4.5	Algorithm	18
4.6	Correctness proof	19
4.7	Complexity	19
4.8	C++ implementation notes	19
4.9	Python implementation notes	19
4.10	Full C++ solution	19
4.11	Full Python solution	20

4.12	Common wrong answers	21
4.13	Tests that matter	21
5	D. Penalty Order	22
5.1	Restatement and observation	22
5.2	Correctness proof	22
5.3	Complexity and implementation notes	22
5.4	Full C++ solution	22
5.5	Full Python solution	22
5.6	Tests that matter	23
6	Stress-testing strategy	24
7	What to carry forward	25
8	ICPC readiness ladder editorials	26
8.1	1. ACM Contest Scoring – simulate the stated rule	26
8.2	2. Booking a Room – first unmarked label	26
8.3	3. Sort of Sorting – key plus stable ties	26
8.4	4. ICPC Team Selection – spend extremes on harmless roles	26
8.5	5. Guess the Data Structure – eliminate candidates independently	27
8.6	6. Assigning Workstations – expire, then reuse	27
8.7	7. Teque – make the middle a boundary	27
8.8	8. Almost Union-Find – moving means creating a new representative	27
8.9	9. Single Source Shortest Path – settle the smallest tentative value	27
8.10	10. Get Shorty – Dijkstra over products	28
8.11	11. Coast Length – flood from guaranteed outside water	28
8.12	12. All Pairs Shortest Path – distinguish finite, unreachable, and unbounded	28
8.13	13. Arbitrage – profitable trade is a negative cycle	28
8.14	14. Knapsack – choose once, then remember predecessors	28
8.15	15. Clock Pictures – rotation is cyclic gap matching	28
8.16	16. String Matching – KMP preserves the longest useful suffix	29
8.17	17. Max Flow – residual reachability certifies optimality	29
8.18	18. Min Cut – read the answer from the final residual graph	29
8.19	19. Distributing Ballot Boxes – binary-search the worst assignment	29
8.20	20. Catering – turn reusable meals into min-cost flow	29
9	Offline Companion Editorials	30
9.1	1. ACM Contest Scoring: local companion	30
9.2	2. Booking a Room: local companion	32
9.3	Restatement	32
9.4	Constraints and traps	32
9.5	Observation and walk-through	32
9.6	Algorithm	32
9.7	Correctness proof	33
9.8	Complexity	33
9.9	Full C++ solution	33
9.10	Full Python solution	33
9.11	Tests that matter	34
9.12	3. Sort of Sorting: local companion	34
9.13	Restatement and constraints	34
9.14	Observation and walk-through	34

9.15	Algorithm	35
9.16	Correctness proof	35
9.17	Complexity and implementation notes	35
9.18	Full C++ solution	35
9.19	Full Python solution	36
9.20	Tests that matter	36
9.21	4. ICPC Team Selection: local companion	36
9.22	Restatement	36
9.23	Observation and trace	36
9.24	Algorithm	37
9.25	Correctness proof	37
9.26	Complexity	37
9.27	Implementation traps and tests	38
10	Stress testing greedy	39
11	What to carry forward	40
11.1	Reference implementations	40
11.2	Full C++ solution	40
11.3	Full Python solution	41
11.4	5. Guess the Data Structure: local companion	42
11.5	Restatement	42
11.6	Observation and trace	42
11.7	Algorithm	42
11.8	Correctness proof	43
11.9	Complexity	43
11.10	Implementation traps and tests	43
11.11	Full C++ solution	44
11.12	Full Python solution	44
12	Stress testing	45
13	What to carry forward	46
13.1	6. Assigning Workstations: local companion	46
13.2	Restatement and traps	46
13.3	Observation and trace	46
13.4	Algorithm	46
13.5	Correctness proof	47
13.6	Complexity	47
13.7	Full C++ solution	47
13.8	Full Python solution	47
13.9	Implementation notes and tests	48
13.10	Stress testing	48
14	What to carry forward	49
14.1	7. Teque: local companion	49
14.2	Restatement and constraint implications	49
14.3	Coordinate collection and trace	49
14.4	Algorithm	49
14.5	Correctness proof	50
14.6	Complexity, implementation notes, and tests	50
14.7	Full C++ solution	50

14.8 Full Python solution	52
15 Testing discussion	54
16 Detailed upsolve workshop	55
16.1 A. Hotel Queries	55
16.2 B. List Removals	55
16.3 C. Mo Distinct Queries	56
17 What to carry forward	57
17.1 8. Almost Union-Find: local companion	57
17.2 Restatement	57
17.3 Observation: keep a minimum spanning forest	57
17.4 Walk-through	58
17.5 Algorithm	58
17.6 Correctness proof	58
17.7 Complexity	59
17.8 Implementation notes	59
17.9 Full C++ solution	59
17.10 Full Python solution	61
17.11 Common wrong answers	62
17.12 Tests that matter	62
18 Stress testing	63
19 What to carry forward	64
19.1 9. Single Source Shortest Path: local companion	64
19.2 Restatement	64
19.3 Constraints and traps	64
19.4 Observation and walk-through	64
19.5 Algorithm	65
19.6 Correctness proof	65
19.7 Complexity	66
19.8 C++ implementation notes	66
19.9 Python implementation notes	66
19.10 Full C++ solution	66
19.11 Full Python solution	67
19.12 Common wrong answers	68
19.13 Tests that matter	68
20 Common wrong answers	69
21 Stress testing	70
22 What to carry forward	71
22.1 10. Get Shorty: local companion	71
22.2 Restatement	71
22.3 Model the implicit graph	71
22.4 Expanding one portal group once	72
22.5 Walk-through	72
22.6 Algorithm	72
22.7 Correctness proof	73
22.8 Complexity	73

22.9 Implementation notes	74
22.10 Full C++ solution	74
22.11 Full Python solution	75
22.12 Common wrong answers	76
22.13 Tests that matter	77
23 Stress testing	78
24 What to carry forward	79
24.1 11. Coast Length: local companion	79
24.2 Restatement	79
24.3 Key idea: start BFS from every station	79
24.4 Walk-through	80
24.5 Algorithm	80
24.6 Correctness proof	80
24.7 Complexity	81
24.8 Reference implementation in C++	81
24.9 Reference implementation in Python	82
24.10 Tests worth checking	83
24.11 12. All Pairs Shortest Path: local companion	83
24.12 Restatement	83
24.13 Threshold viewpoint	83
24.14 Walk-through	83
24.15 Algorithm	84
24.16 Correctness proof	84
24.17 Complexity	85
24.18 Implementation notes	85
24.19 Full C++ solution	85
24.20 Full Python solution	86
24.21 Common wrong answers	87
24.22 Tests that matter	87
25 Stress testing	88
26 What to carry forward	89
26.1 13. Arbitrage: local companion	89
26.2 Restatement and observation	89
26.3 Constraints and traps	89
26.4 Worked trace	90
26.5 Algorithm	90
26.6 Correctness proof	90
26.7 Complexity	90
26.8 Full C++ solution	90
26.9 C++ implementation notes	91
26.10 Full Python solution	91
26.11 Python implementation notes	92
26.12 Traps and tests	92
27 Stress testing	93
28 What to carry forward	94
28.1 Constraints and traps	94
28.2 Reachability trace	95

28.3	Algorithm	95
28.4	Correctness proof	95
28.5	Complexity	96
28.6	Full C++ solution	96
28.7	C++ implementation notes	96
28.8	Full Python solution	97
28.9	Python implementation notes	97
28.10	Common wrong answers	97
28.11	Tests that matter	97
29	Stress testing	98
30	What to carry forward	99
30.1	15. Clock Pictures: local companion	99
30.2	Restatement and traps	99
30.3	Observation and trace	99
30.4	Correctness proof	100
30.5	Complexity	100
30.6	Full C++ solution	100
30.7	Full Python solution	100
30.8	Tests and stress testing	101
30.9	16. String Matching: local companion	101
30.10	Restatement	101
30.11	Constraints and traps	101
30.12	Observations and trace	101
30.13	Algorithm	101
30.14	Correctness proof	102
30.15	Complexity	102
30.16	C++ implementation notes	102
30.17	Python implementation notes	102
30.18	Full C++ solution	103
30.19	Full Python solution	103
30.20	Common wrong answers	103
30.21	Tests that matter	104
31	What to carry forward	105
32	Testing discussion	106
32.1	17. Max Flow: local companion	106
32.2	Restatement and reduction target	106
32.3	Network construction and trace	106
32.4	Algorithm	107
32.5	Correctness proof	107
32.6	Complexity, implementation notes, and tests	107
32.7	Full C++ solution	107
32.8	Full Python solution	109
33	What to carry forward	112
34	Testing discussion	113
34.1	18. Min Cut: local companion	113
34.2	Restatement and reduction target	113
34.3	Network construction and trace	113

34.4	Algorithm	114
34.5	Correctness proof	114
34.6	Complexity, implementation notes, and tests	114
34.7	Full C++ solution	114
34.8	Full Python solution	116
35	What to carry forward	119
36	Testing discussion	120
36.1	19. Distributing Ballot Boxes: local companion	120
36.2	Restatement and predicate	120
36.3	Bounds, trace, and algorithm	120
36.4	Correctness proof	121
36.5	Complexity and implementation notes	121
36.6	Full C++ solution	121
36.7	Full Python solution	122
36.8	20. Catering: local companion	123
36.9	State expansion	123
36.10	Correctness proof	123
36.11	Complexity and tests	123
36.12	Full C++ solution	123
36.13	Full Python solution	124
37	What to carry forward	125

1 How to read this editorial

These problems are deliberately simpler than a World Finals set. Their value is accuracy under contest conditions: parse the rule into state, identify a known tool despite an unfamiliar story, and prove the exact operation that makes the implementation safe.

2 Problem A: Scoreboard Replay

2.1 Restatement

Replay submissions under the problem's stated scoring rule. A team's first accepted submission for a problem increases its solved count. Its penalty for that problem is the accepted minute plus 20 for each earlier wrong submission. Later submissions to that solved problem are ignored. Finally, sort teams by more solved problems, then lower penalty, then lower id.

2.2 Important constraints and traps

The chronological order in the input matters. Wrong submissions do not add to the total penalty until that problem is accepted. A wrong submission after an acceptance has no effect. These two details are the source of most incorrect solutions.

2.3 Observation

The only information needed for one pair (`team`, `problem`) is:

```
solved[team][problem]
wrong[team][problem]
```

The pair's future behavior depends only on those two values, not on the full history. Maintain team-wide `solved_count` and `penalty` at the moment of the first acceptance.

2.4 Sample walk-through

Suppose team 2 submits W at minute 18, W at 31, and A at 47 for the same problem. The first two submissions increase `wrong[2][p]` from 0 to 2. The acceptance marks it solved, increments the solved count, and adds $47 + 20 * 2 = 87$ penalty. Any later record for (2, p) is ignored.

2.5 Algorithm

```

initialize solved, wrong, solved_count, penalty to zero
for each submission (minute, team, problem, verdict):
    if solved[team][problem]:
        continue
    if verdict is W:
        wrong[team][problem] += 1
    else:
        solved[team][problem] = true
        solved_count[team] += 1
        penalty[team] += minute + 20 * wrong[team][problem]
sort team ids by (-solved_count, penalty, id)
print each id with its final score

```

2.6 Correctness proof

We prove that the algorithm prints the required final ranking.

For a fixed team and problem, after processing any prefix of submissions, **solved** is true exactly when the prefix contains an acceptance before any later considered submission, and **wrong** equals the number of wrong submissions before that first acceptance if it has not happened yet. Initially both statements hold. A record after acceptance is correctly ignored. Before acceptance, a wrong record increases exactly that count; an accepted record adds exactly the required penalty, marks the pair solved, and prevents every later record from changing it. Thus the invariant holds by induction.

Consequently each first acceptance contributes one solved problem and exactly its stated penalty, while every other submission contributes nothing. The stored score for every team is therefore correct. The final comparator is the problem's ranking rule, so sorting by it prints the required order. $\square$

2.7 Complexity

The replay uses $O(S)$ time and $O(TP)$ memory for S submissions, T teams, and P problems. Sorting the teams costs $O(T \log T)$ time.

2.8 C++ implementation notes

Use `vector<vector<bool>>` or `vector<vector<char>>` for **solved** and `vector<vector<int>>` for **wrong** attempts. The comparator should return a tuple such as `(-solved_count[id], penalty[id], id)`; do not try to encode all ties with nested conditionals.

2.9 Python implementation notes

Nested lists make the pair state clear. `sorted(range(1, teams + 1), key=lambda team: (-solved_count[team], penalty[team], team))` implements the ranking rule directly.

2.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int teams, problems, submissions;
    if (!(cin >> teams >> problems >> submissions)) return 0;
    vector<vector<char>> solved(teams + 1, vector<char>(problems + 1));
    vector<vector<int>> wrong(teams + 1, vector<int>(problems + 1));
    vector<int> solved_count(teams + 1);
    vector<long long> penalty(teams + 1);

    while (submissions--) {
        int minute, team, problem;
        char verdict;
        cin >> minute >> team >> problem >> verdict;
        if (solved[team][problem]) continue;
        if (verdict == 'W') {
            ++wrong[team][problem];
        } else {
            solved[team][problem] = true;
            ++solved_count[team];
            penalty[team] += minute + 20LL * wrong[team][problem];
        }
    }

    vector<int> order(teams);
    iota(order.begin(), order.end(), 1);
    sort(order.begin(), order.end(), [&](int a, int b) {
        return tuple{-solved_count[a], penalty[a], a} <
            tuple{-solved_count[b], penalty[b], b};
    });
    for (int team : order) {
        cout << team << ' ' << solved_count[team] << ' ' << penalty[team] << '\n';
    }
}
```

2.11 Full Python solution

```
import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
```

```

if not data:
    return
it = iter(data)
teams = int(next(it))
problems = int(next(it))
submissions = int(next(it))
solved = [[False] * (problems + 1) for _ in range(teams + 1)]
wrong = [[0] * (problems + 1) for _ in range(teams + 1)]
solved_count = [0] * (teams + 1)
penalty = [0] * (teams + 1)

for _ in range(submissions):
    minute = int(next(it))
    team = int(next(it))
    problem = int(next(it))
    verdict = next(it)
    if solved[team][problem]:
        continue
    if verdict == b"W":
        wrong[team][problem] += 1
    else:
        solved[team][problem] = True
        solved_count[team] += 1
        penalty[team] += minute + 20 * wrong[team][problem]

order = sorted(range(1, teams + 1), key=lambda team: (-solved_count[team], penalty[team]))
print("\n".join(f"{team} {solved_count[team]} {penalty[team]}" for team in order))

if __name__ == "__main__":
    main()

```

2.12 Common wrong answers

- Add 20 penalty for every wrong submission, including unsolved problems.
- Count an acceptance twice when duplicate accepted records appear.
- Sort equal scores by input order rather than team id.

2.13 Tests that matter

Use a team with no submissions, several wrong submissions before one acceptance, a wrong submission after acceptance, tied teams, and multiple problems for one team.

3 Problem B: Time-Boxed Schedule

3.1 Restatement

Each task takes a positive number of minutes and has a deadline. Starting at minute zero, complete at most one task at a time. Find the maximum number of tasks that can all finish no later than their own deadlines.

3.2 Important constraints and traps

The target is the number of tasks, not the smallest total duration. A task is allowed to finish exactly at its deadline. The chosen tasks must be schedulable in deadline order; arbitrary input order is not meaningful.

3.3 Observation

Sort by nondecreasing deadline. When considering a task, tentatively keep it. If the total selected duration exceeds its deadline, the selected set is no longer feasible by this deadline. To preserve as many tasks as possible, discard the longest selected task: this removes one task but leaves the least possible total time among all choices of that size.

3.4 Sample walk-through

For (duration, deadline) equal to (4, 4), (3, 5), (2, 6), first keep 4. After adding 3, the total 7 exceeds deadline 5, so remove the longest duration 4 and keep only 3. Adding 2 gives total 5 by deadline 6, so two tasks remain. Choosing the first two tasks would complete only one on time; the heap makes the replacement explicit.

3.5 Algorithm

```
sort tasks by deadline
total = 0
max_heap = empty
for (duration, deadline) in sorted tasks:
    add duration to total and max_heap
    if total > deadline:
        total -= pop largest duration from max_heap
print heap size
```

3.6 Correctness proof

After processing every task whose deadline is at most d , maintain this invariant: the heap contains a feasible subset of maximum cardinality among processed tasks, and among maximum-cardinality feasible subsets it has minimum total duration.

Before the first task the invariant is immediate. Add the next task with deadline d . If the new total is at most d , the old set plus this task is feasible. No subset of processed tasks can have more than one additional task than the old maximum, so the new set has maximum cardinality; it is the only new cardinality candidate and preserves the invariant. If the total exceeds d , every feasible subset of this new size must remove at least one task. By removing a longest duration, the algorithm leaves the minimum possible total duration among all subsets formed by removing one task. If even that total were not feasible, no subset of this size could be; otherwise the remaining set is feasible and has the old maximum cardinality. The invariant holds.

At the end, the heap has maximum feasible cardinality among all tasks, which is the required answer. $\square$

3.7 Complexity

Sorting takes $O(N \log N)$. Each task is inserted and removed from the heap at most once, for $O(N \log N)$ total time and $O(N)$ memory.

3.8 C++ implementation notes

`priority_queue<long long>` is a max heap. Store the running total in `long long` because many individually small durations can sum beyond `int`.

3.9 Python implementation notes

`heapq` is a min heap. Push `-duration` and add `heapq.heappop(heap)` to the total when removing, because the popped value is negative.

3.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<pair<long long, long long>> tasks(n);
```

```

for (auto& [duration, deadline] : tasks) cin >> duration >> deadline;
sort(tasks.begin(), tasks.end(), [](auto a, auto b) { return a.second < b.second; });

priority_queue<long long> chosen;
long long total = 0;
for (auto [duration, deadline] : tasks) {
    total += duration;
    chosen.push(duration);
    if (total > deadline) {
        total -= chosen.top();
        chosen.pop();
    }
}
cout << chosen.size() << '\n';
}

```

3.11 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    tasks = [(data[i], data[i + 1]) for i in range(1, 2 * n + 1, 2)]
    tasks.sort(key=lambda task: task[1])
    chosen = []
    total = 0
    for duration, deadline in tasks:
        total += duration
        heapq.heappush(chosen, -duration)
        if total > deadline:
            total += heapq.heappop(chosen)
    print(len(chosen))

if __name__ == "__main__":
    main()

```

3.12 Common wrong answers

- Keep the shortest task only when a deadline is exceeded; the correct action is to remove the longest selected task.
- Sort by duration instead of deadline.

- Treat `total == deadline` as infeasible.

3.13 Tests that matter

Use one impossible task, equal deadlines, a case where replacing one long task by several short tasks is necessary, and durations whose sum needs 64 bits.

4 Problem C: First-Round Assignment

4.1 Restatement

There are teammates and unsolved problem cards. An edge means that a teammate can take ownership of that card in the first independent round. Each teammate and each card may be used at most once. Find the maximum number of cards that can receive distinct owners.

4.2 Important constraints and traps

This is not simply the number of teammates with any edge. Two teammates may both only be able to own the same card. Conversely, a teammate can be moved from an already matched card along an alternating path to make room for a new card.

4.3 Observation

Make a bipartite graph: teammates on the left and cards on the right. A valid first-round plan is exactly a matching. Use the augmenting-path DFS introduced in Section 45.

4.4 Sample walk-through

Let teammate 1 connect to cards 1 and 2, teammate 2 connect only to card 1, and teammate 3 connect to card 2 and 3. Match teammate 1 to card 1. When trying teammate 2, card 1 is occupied, but teammate 1 can move to card 2. This alternating path gives teammate 2 card 1. Teammate 3 then takes card 3, so all three receive distinct cards.

4.5 Algorithm

```
match_card[c] = no teammate for every card
answer = 0
for each teammate u:
    clear seen_card
    if augment(u):
        answer += 1

augment(u):
    for each card c adjacent to u:
        if seen_card[c]: continue
```

```

    mark seen_card[c]
    if match_card[c] is empty or augment(match_card[c]):
        match_card[c] = u
        return true
    return false

```

4.6 Correctness proof

At every point `match_card` describes a matching because a card stores at most one teammate and a teammate is reassigned only as part of a recursive alternating path. If `augment(u)` succeeds, the path starts at unmatched teammate `u` and ends at an unmatched card; reversing its matched and unmatched edges increases the matching size by one while maintaining validity.

If `augment(u)` fails, the DFS has explored every card reachable from `u` by alternating paths, so no augmenting path starting at `u` exists. By Berge's augmenting-path characterization, the current matching cannot be enlarged via that unmatched teammate. Processing all teammates therefore produces a maximum matching. Since matchings are exactly valid first-round assignments, the reported size is optimal. $\square$

4.7 Complexity

With L teammates and E compatibility edges, this DFS implementation takes $O(L \cdot E)$ time and $O(L + R)$ auxiliary memory, where R is the number of cards.

4.8 C++ implementation notes

Clear `seen` for each outer DFS, not once for the whole algorithm. Capturing the recursive lambda with `std::function<bool(int)>` is concise and adequate for these bounds.

4.9 Python implementation notes

The same `seen` reset is essential. Raise the recursion limit only when the input bounds justify it; the local task is deliberately small enough for a direct DFS.

4.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int teammates, cards, edges;
if (!(cin >> teammates >> cards >> edges)) return 0;
vector<vector<int>> graph(teammates + 1);
while (edges--) {
    int teammate, card;
    cin >> teammate >> card;
    graph[teammate].push_back(card);
}

vector<int> match_card(cards + 1);
vector<char> seen(cards + 1);
function<bool(int)> augment = [&](int teammate) {
    for (int card : graph[teammate]) {
        if (seen[card]) continue;
        seen[card] = true;
        if (match_card[card] == 0 || augment(match_card[card])) {
            match_card[card] = teammate;
            return true;
        }
    }
    return false;
};

int answer = 0;
for (int teammate = 1; teammate <= teammates; ++teammate) {
    fill(seen.begin(), seen.end(), false);
    answer += augment(teammate);
}
cout << answer << '\n';
}

```

4.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    teammates, cards, edges = data[:3]
    graph = [[] for _ in range(teammates + 1)]
    index = 3
    for _ in range(edges):
        teammate, card = data[index], data[index + 1]
        index += 2
        graph[teammate].append(card)

    match_card = [0] * (cards + 1)

```

```

def augment(teammate: int) -> bool:
    for card in graph[teammate]:
        if seen[card]:
            continue
        seen[card] = True
        if match_card[card] == 0 or augment(match_card[card]):
            match_card[card] = teammate
            return True
    return False

answer = 0
for teammate in range(1, teammates + 1):
    seen = [False] * (cards + 1)
    answer += augment(teammate)
print(answer)

if __name__ == "__main__":
    main()

```

4.12 Common wrong answers

- Assign each teammate greedily to its first available card.
- Forget that a currently matched teammate may move to another card.
- Keep `seen` from the previous teammate's search.

4.13 Tests that matter

Use no edges, more teammates than cards, duplicate edges, and an augmenting path that needs to rematch at least two teammates.

5 D. Penalty Order

5.1 Restatement and observation

All given problems must be solved sequentially. A problem's penalty is its completion time, and the objective is the sum of all completion times. The problem labels do not matter; only their durations do.

Sort durations in nondecreasing order and add their running prefix sums. For 3,1,2,4, the sorted order has completion times 1,3,6,10, so the answer is 20.

5.2 Correctness proof

Consider two consecutive durations a then b with $a > b$, after an already elapsed time E . Their contribution is $(E+a) + (E+a+b)$. Swapping them gives $(E+b) + (E+b+a)$, which is smaller by $a-b$. Thus any inversion can be swapped away without increasing the answer. Repeated swaps transform an optimum into nondecreasing order, so sorting by duration is optimal.

5.3 Complexity and implementation notes

Sorting costs $O(n \log n)$ time and the scan is $O(n)$. The maximum total is $n(n+1)/2 * 10^9$, which can exceed signed 64-bit at these bounds. Use `__int128` in C++ and print it manually; Python integers already grow as needed.

5.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>n))return 0;vector<long long>
```

5.5 Full Python solution

```
import sys
def main() -> None:
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data:return
    elapsed=answer=0
    for duration in sorted(data[1:]):
```

```
        elapsed+=duration;answer+=elapsed
    print(answer)
if __name__=='__main__':main()
```

5.6 Tests that matter

One problem.
Already sorted and reverse-sorted durations.
Equal durations.
Very large durations, which require a wide integer type.

The tiny oracle checks every permutation. The scale case has 200000 distinct durations with a direct arithmetic expected answer.

6 Stress-testing strategy

For A, compare the state simulation to a slow per-team/per-problem replay on small random histories. For B, enumerate every subset for $N \leq 18$, sort each subset by deadline, and compare its feasible count. For C, enumerate all assignments on very small bipartite graphs or compare with a max-flow model. The oracle must be independently simpler than the optimized implementation.

7 What to carry forward

The final competitive advantage is not a magical template. It is a team that can make its model, invariant, blocker, and next test visible. Repeat the simulation protocol in `PRACTICE.md`, then use its postmortem to choose the next individual training block.

8 ICPC readiness ladder editorials

These twenty external tasks are ordered from dependable opening solves to finalist-style reductions. For each task, recover the state representation, the invariant, and the reason the target complexity fits before consulting an implementation.

8.1 1. ACM Contest Scoring – simulate the stated rule

For every problem, store its number of earlier wrong submissions and whether it has already been solved. A wrong answer increments only the former; the first acceptance contributes its minute and the penalty for those earlier wrongs; later records for that problem do nothing. Never sort the records. The prefix invariant is exact: these two fields describe the contest state after every processed submission. Test repeated wrong answers, acceptance, and a later submission to the solved problem.

8.2 2. Booking a Room – first unmarked label

Mark reserved rooms in a boolean array and scan room numbers upward. The first unmarked index is the smallest available room; if no such index exists, all rooms are booked. This is $O(R + B)$ and the scan order itself proves the tie rule. Test the first room, last room, and all-booked cases.

8.3 3. Sort of Sorting – key plus stable ties

Sort names by the two-character key requested by the statement. Equal keys must retain input order, so use a stable sort or original index as a secondary key. Comparing whole names is a different ordering. Stable sorting orders all different keys and preserves each equal-key class, which proves correctness. The blank-line convention between datasets is part of the output contract.

8.4 4. ICPC Team Selection – spend extremes on harmless roles

Sort the $3n$ skills. A team score is its median, so smallest values can serve as low members and largest values can serve as high members without increasing the chosen median. Take medians from the upper eligible region while consuming one largest value for each team. An exchange argument shows that replacing a chosen median with a smaller unused value cannot help, while a larger unused value can still act as the high member. Test equal values and a single team.

8.5 5. Guess the Data Structure – eliminate candidates independently

Simulate stack, queue, and max-priority queue in parallel. On a removal, kill a candidate if it is empty or predicts a different value. After all operations, zero survivors means impossible, one identifies the type, and several mean “not sure.” The invariant is independent consistency with the full processed prefix. Do not commit after the first removal: short traces often fit all three.

8.6 6. Assigning Workstations – expire, then reuse

Process arrivals in time order and store workstation release times in a min-heap. Remove entries whose unlocked period expired before this arrival. If the smallest surviving release is reusable, consume it; otherwise unlock a new machine. Choosing the earliest reusable release is safe because it expires first and leaves all later choices available. Check the statement’s exact endpoint convention for a timeout equal to the next arrival.

8.7 7. Teque – make the middle a boundary

Use two deques whose concatenation is the sequence, keeping their sizes equal or the left deque one larger. The size invariant tells exactly where a middle insertion belongs and gives $O(1)$ indexed lookup by choosing a deque first. After each push, rebalance by moving one boundary item when necessary. This avoids linear insertion into an ordinary array.

8.8 8. Almost Union-Find – moving means creating a new representative

DSU cannot delete an element. Give each element a current private node; when moving it, subtract it from the old root’s size and sum, create a fresh node for it, then union that node into the destination component. Old nodes may stay in the forest but no live element points to them. This invariant preserves standard path compression and union-by-size. Use 64-bit sums and test moving within the same set.

8.9 9. Single Source Shortest Path – settle the smallest tentative value

With nonnegative weights, Dijkstra’s heap stores tentative distances. When a non-stale entry is popped, no unseen route can improve it: any competing route must cross the settled boundary through a nonnegative edge and would have created an earlier tentative entry. Relax outgoing edges, ignore stale heap entries, and print **Impossible** only for infinite distances. Use 64-bit costs.

8.10 10. Get Shorty – Dijkstra over products

Store the largest product reachable at each vertex in a max-heap. Relax an edge when `best[u] * factor` improves `best[v]`. Factors are at most one, so extending a path cannot improve its product; therefore popping the largest candidate settles a vertex. Equivalently, `-log(factor)` converts the task to ordinary nonnegative shortest paths. Never add the factors.

8.11 11. Coast Length – flood from guaranteed outside water

Pad the grid by one water border and flood-fill from that exterior water. Each adjacent land edge seen by the flood is coastline. Enclosed water is never visited, so lake boundaries are excluded; every ocean-facing land edge is seen from its exterior neighbor. Padding eliminates special boundary handling.

8.12 12. All Pairs Shortest Path – distinguish finite, unreachable, and unbounded

Run Floyd-Warshall. Its phase invariant allows only the first `k` vertices as internal vertices after phase `k`. A query is unreachable when its distance is infinite. It is `-Infinity` when the source can reach some `k` with `dist[k][k] < 0` and that `k` can reach the destination. Otherwise output the finite distance. Guard infinity before adding distances.

8.13 13. Arbitrage – profitable trade is a negative cycle

A cycle is profitable precisely when its exchange-rate product exceeds one. Taking negative logarithms turns products into sums and such a product into a negative cycle. Detect one with Bellman-Ford or Floyd-Warshall. The reduction is exact because logarithm preserves multiplication as addition and negation reverses the comparison around one. Avoid floating-point equality tests near one; use the problem's prescribed comparison tolerance if any.

8.14 14. Knapsack – choose once, then remember predecessors

For each capacity, store the best value using processed items. Iterate capacities downward for a 0/1 item so it cannot be reused in the same round. To print a witness, retain the decisions or predecessor capacities whenever a state improves. The item-prefix induction proves the include/exclude transition is complete. Backtracking the recorded choices reconstructs one optimal set.

8.15 15. Clock Pictures – rotation is cyclic gap matching

Sort both angle sets and replace each by its cyclic sequence of consecutive gaps, including the wrap-around gap. Rotation changes only the starting point; it preserves the cyclic gap sequence, and equal sequences reconstruct equal rotated pictures. Search one gap list in the doubled other list with KMP. Repeated gaps are why a greedy alignment is unsafe.

8.16 16. String Matching – KMP preserves the longest useful suffix

The prefix function records, after matching a prefix length j , the next largest prefix that can still match after a mismatch. During the text scan, maintain that j is the longest pattern prefix matching a suffix of the text already read. On a full match, record its start and jump through the failure link, which preserves overlaps. Construction and scanning are both linear.

8.17 17. Max Flow – residual reachability certifies optimality

Create a reverse residual edge for every original edge. Augment source-to-sink paths, updating both directions; Dinic finds blocking flows efficiently. The flow remains feasible by conservation. When no residual path remains, vertices reachable from the source define a cut whose outgoing edges are saturated, and its capacity equals the current flow. That cut certifies maximality.

8.18 18. Min Cut – read the answer from the final residual graph

After maximum flow, BFS from the source through positive-residual edges. The visited set is the source side of a minimum cut: no residual edge can leave it, so all original outgoing cut edges are saturated. Reverse residual capacity shows the flow value equals that cut capacity. No second cut algorithm is needed.

8.19 19. Distributing Ballot Boxes – binary-search the worst assignment

For a candidate maximum population per box x , city population p needs $\text{ceil}(p/x)$ boxes. Sum those needs and test whether they fit the available box count. The predicate is monotone because increasing x cannot require more boxes. Binary-search the first feasible x , with integer ceiling division $(p+x-1)/x$. Test exact divisibility and one extra box.

8.20 20. Catering – turn reusable meals into min-cost flow

Model every meal as a unit of flow from purchase or an earlier event to a later event that consumes it. Source edges represent new meals; directed reuse edges represent cleaning and transport; event capacities represent demand. Every physical meal history maps to one such path, and every integral flow describes that many meals, so minimum-cost flow gives the optimum. Compare a two-event case where reuse is cheaper with one where buying anew is cheaper.

9 Offline Companion Editorials

The linked contest tasks above are external judge problems. The following entries are the full editorials for their original local companions. Each contains a complete C++ and Python reference implementation, a correctness argument, complexity analysis, and hostile-test guidance. The mapping is deliberate: use the companion to rehearse the course skill offline, then return to the linked contest task without treating the companion as a replacement for its statement.

9.1 1. ACM Contest Scoring: local companion

External Kattis task: <https://open.kattis.com/problems/acm> Local package: [problems/01_acm_contest_scoring](#)

The detailed scoreboard derivation appears earlier in this editorial. The two references below are repeated here so every companion entry is independently usable from the offline PDF.

9.1.1 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int teams, problems, submissions;
    if (!(cin >> teams >> problems >> submissions)) return 0;
    vector<vector<char>> solved(teams + 1, vector<char>(problems + 1));
    vector<vector<int>> wrong(teams + 1, vector<int>(problems + 1));
    vector<int> solved_count(teams + 1);
    vector<long long> penalty(teams + 1);

    while (submissions--) {
        int minute, team, problem;
        char verdict;
        cin >> minute >> team >> problem >> verdict;
        if (solved[team][problem]) continue;
        if (verdict == 'W') {
            ++wrong[team][problem];
        } else {
            solved[team][problem] = true;
            ++solved_count[team];
            penalty[team] += minute + 20LL * wrong[team][problem];
        }
    }
}
```

```

    }
}

vector<int> order(teams);
iota(order.begin(), order.end(), 1);
sort(order.begin(), order.end(), [&](int a, int b) {
    return tuple{-solved_count[a], penalty[a], a} <
           tuple{-solved_count[b], penalty[b], b};
});
for (int team : order) {
    cout << team << ' ' << solved_count[team] << ' ' << penalty[team] << '\n';
}
}

```

9.1.2 Full Python solution

```

import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = iter(data)
    teams = int(next(it))
    problems = int(next(it))
    submissions = int(next(it))
    solved = [[False] * (problems + 1) for _ in range(teams + 1)]
    wrong = [[0] * (problems + 1) for _ in range(teams + 1)]
    solved_count = [0] * (teams + 1)
    penalty = [0] * (teams + 1)

    for _ in range(submissions):
        minute = int(next(it))
        team = int(next(it))
        problem = int(next(it))
        verdict = next(it)
        if solved[team][problem]:
            continue
        if verdict == b"W":
            wrong[team][problem] += 1
        else:
            solved[team][problem] = True
            solved_count[team] += 1
            penalty[team] += minute + 20 * wrong[team][problem]

    order = sorted(range(1, teams + 1), key=lambda team: (-solved_count[team], penalty[team]))
    print("\n".join(f"{team} {solved_count[team]} {penalty[team]}" for team in order))

```

```
if __name__ == "__main__":
    main()
```

9.2 2. Booking a Room: local companion

External Kattis task: <https://open.kattis.com/problems/bookingaroom> Local package: [problems/02_booking_a_room](#)

9.3 Restatement

Given closed busy intervals within $0..limit$ and a required integer duration d , find the smallest start $s \geq 0$ whose entire interval $[s, s+d-1]$ is not busy. Return -1 if no such start exists.

9.4 Constraints and traps

The total number of intervals is at most 200000, so testing every possible start is impossible. Intervals are closed: if one ends at r , the earliest time after it is $r+1$, not r . Input intervals may overlap, nest, or touch, and $1 \leq r$ is guaranteed. Use `long long` for $s+d-1$.

9.5 Observation and walk-through

Sort closed busy intervals and maintain `cur`, the earliest time not already blocked by processed intervals. If $cur+d-1 < l$, the gap before the next interval fits and `cur` is the answer. Otherwise merge the obstacle by setting $cur = \max(cur, r+1)$. After all intervals, accept only when $cur+d-1 \leq limit$.

For busy intervals $[1,2], [5,7], [7,9]$, duration three, and limit twelve, start `cur=0`. The initial candidate $[0,2]$ overlaps $[1,2]$, so move to 3. $[3,5]$ overlaps $[5,7]$, so move to 8; $[8,10]$ overlaps $[7,9]$, so move to 10. Finally $[10,12]$ is free, so the answer is 10.

9.6 Algorithm

```
sort intervals by left endpoint
cur = 0
for [l, r] in sorted intervals:
    if cur + d - 1 < l: return cur
    cur = max(cur, r + 1)
return cur if cur + d - 1 <= limit else -1
```

9.7 Correctness proof

The invariant is that no start below `cur` can still begin a legal slot after the processed intervals. Sorting makes every future interval start no earlier than the current one, so a fitting gap can be returned immediately. Each interval either blocks the current candidate and advances `cur` past its right endpoint, or leaves a gap large enough to return the smallest possible start. After all intervals, the final limit test accepts exactly the remaining legal candidate. Thus the algorithm returns the earliest valid start or `-1`.

9.8 Complexity

Sorting costs $O(n \log n)$, the scan is linear, and memory is $O(n)$.

9.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests; cin >> tests;
    while (tests--) {
        int n; long long duration, limit;
        cin >> n >> duration >> limit;
        vector<pair<long long, long long>> busy(n);
        for (auto &interval : busy) cin >> interval.first >> interval.second;
        sort(busy.begin(), busy.end());
        long long cur = 0;
        for (auto [left, right] : busy) {
            if (cur + duration - 1 < left) break;
            cur = max(cur, right + 1);
        }
        cout << (cur + duration - 1 <= limit ? cur : -1) << '\n';
    }
}
```

9.10 Full Python solution

```
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
index = 1
out = []
for _ in range(data[0]):
    n, duration, limit = data[index:index + 3]
```

```

index += 3
busy = sorted((data[i], data[i + 1]) for i in range(index, index + 2 * n, 2))
index += 2 * n
cur = 0
for left, right in busy:
    if cur + duration - 1 < left:
        break
    cur = max(cur, right + 1)
out.append(str(cur if cur + duration - 1 <= limit else -1))
print("\n".join(out))

```

9.11 Tests that matter

Test no intervals, touching intervals such as [2,2] and [3,3], nested intervals, and a valid slot ending exactly at `limit`. The small oracle scans integer starts; the scale case checks the sorted scan is necessary.

9.12 3. Sort of Sorting: local companion

External Kattis task: <https://open.kattis.com/problems/sortofsorting> Local package: [problems/03_sort_of_sorting](#)

9.13 Restatement and constraints

Choose exactly k values whose maximum minus minimum is as small as possible. With total n up to 200000, enumerating all k -subsets is impossible even though it is a useful tiny-input oracle. Values may be negative, and the answer can exceed 32-bit range.

9.14 Observation and walk-through

Sort the values. In an optimal chosen team, consider its smallest and largest chosen values. Every sorted value between them can replace an interior chosen value without increasing the spread, so some optimum is a consecutive block of exactly k sorted values.

For [4,1,9,7,5] and $k=3$, sorting gives [1,4,5,7,9]:

```

[1,4,5] spread 4
[4,5,7] spread 3
[5,7,9] spread 4

```

The answer is three.

9.15 Algorithm

```
sort a
answer = a[k-1] - a[0]
for left from 1 through n-k:
    answer = min(answer, a[left+k-1] - a[left])
print answer
```

9.16 Correctness proof

Let S be an optimal team and let its minimum and maximum values occupy sorted positions L and R . The interval $[L, R]$ contains at least k values because it contains S . Choose any k consecutive values inside that interval including its endpoints when necessary; their minimum is no smaller than S 's minimum and their maximum is no larger than S 's maximum. Their spread is therefore at most the optimal spread, hence equal to it. The algorithm evaluates every consecutive block of k sorted values, including this optimal block, and returns the smallest evaluated spread. Thus it is correct.

9.17 Complexity and implementation notes

Sorting costs $O(n \log n)$ and the scan costs $O(n)$; memory is $O(n)$. Use `long long` in C++ for subtraction. Python integers are exact. For $k=1$, every block has spread zero and the same loop handles it.

9.18 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int t; cin >> t;
    while (t--) {
        int n, k; cin >> n >> k;
        vector<long long> a(n);
        for (auto& x : a) cin >> x;
        sort(a.begin(), a.end());
        long long answer = a[k - 1] - a[0];
        for (int left = 1; left + k <= n; ++left)
            answer = min(answer, a[left + k - 1] - a[left]);
        cout << answer << '\n';
    }
}
```

9.19 Full Python solution

```
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
index, out = 1, []
for _ in range(data[0]):
    n, k = data[index], data[index + 1]
    index += 2
    a = sorted(data[index:index + n])
    index += n
    out.append(str(min(a[i + k - 1] - a[i] for i in range(n - k + 1))))
print("\n".join(out))
```

9.20 Tests that matter

Test $k=1$, $k=n$, all equal values, negative values, duplicate values, and two values whose difference exceeds signed 32-bit range. The random generator uses combinations as an independent small oracle and includes a 200000-value case to enforce the intended sorting complexity.

9.21 4. ICPC Team Selection: local companion

External Kattis task: <https://open.kattis.com/problems/icpcteamselection> Local package: [problems/04_icpc_team_selection](https://open.kattis.com/problems/04_icpc_team_selection)

9.22 Restatement

Choose as few integer pickup days as possible so that every attendee interval contains at least one chosen day. The intervals are inclusive: an attendee available from day 2 through day 2 can only be covered on day 2.

This is the section's blind-practice exercise. The output asks for a minimum number of choices, but the safe local choice must still be justified by an exchange argument.

9.23 Observation and trace

Sort intervals by increasing right endpoint. Consider the first interval not already covered, $[l, r]$. Any feasible solution must choose some day d in that interval. Replacing d with r cannot uncover an interval that ends no earlier than r : every remaining interval has right endpoint at least r , and any interval containing d has left endpoint at most $d \leq r$.

So choose day r and continue with intervals it does not cover.

For these intervals:

```
[1,3], [2,5], [4,6], [6,6]
```

the first choice is day 3, covering the first two intervals. The next uncovered interval ends at day 6; choosing day 6 covers both remaining intervals. The answer is 2.

9.24 Algorithm

```
sort intervals by (right endpoint, left endpoint)
last_day = no chosen day
answer = 0

for [left, right] in sorted intervals:
    if no day has been chosen yet or last_day < left:
        choose right
        last_day = right
        answer += 1

print answer
```

An interval with $\text{left} \leq \text{last_day}$ is already covered, because sorted order also ensures last_day is no greater than that interval's right endpoint.

9.25 Correctness proof

Lemma 1. Let $[l, r]$ be the earliest-finishing interval not covered by previous choices. There is an optimal completion that chooses day r for this interval.

Proof. Any feasible completion chooses a day d with $l \leq d \leq r$ to cover $[l, r]$. Replace that choice by r . Every interval still awaiting coverage ends at least at r , because $[l, r]$ has the earliest right endpoint. If such an interval contained d , its left endpoint is at most d , hence at most r ; together with its right endpoint at least r , it contains r . Thus the replacement keeps all intervals covered and does not increase the number of days. $\square$

Lemma 2. Whenever the algorithm chooses a day, it can be part of an optimal solution for all intervals not yet processed.

Proof. The selected interval is the earliest-finishing uncovered interval, so Lemma 1 applies. $\square$

Theorem. The algorithm prints the minimum number of pickup days.

Proof. Repeatedly applying Lemma 2 fixes one choice that agrees with an optimal completion. The algorithm skips exactly the intervals covered by those fixed choices and continues until all intervals are covered. Therefore it uses the same number of pickup days as some optimum. $\square$

9.26 Complexity

Sorting costs $O(n \log n)$ time. The scan costs $O(n)$ time and uses $O(1)$ extra memory beyond the interval list.

9.27 Implementation traps and tests

- Intervals are inclusive: use `last_day < left` to detect an uncovered one.
- Sort by right endpoint, not by left endpoint or interval length.
- Negative day numbers are valid; do not use `-1` as an unguarded sentinel.
- Several nested intervals need only one pickup day.

Test nested intervals, disjoint single-day intervals, intervals sharing an endpoint, and negative endpoints. The random generator enumerates subsets of right endpoints on small instances and includes a 200000-interval scale case.

10 Stress testing greedy

Greedy solutions are excellent candidates for brute-force stress testing on small inputs.

For activity selection:

```
Try every subset of intervals.  
Check whether it is non-overlapping.  
Take the largest valid subset.
```

For rescue boats:

```
Use recursion or DP over remaining people.  
Try sending the heaviest alone or with each possible partner.
```

For deadline scheduling:

```
Try every subset.  
Sort the subset by deadline.  
Check whether every completion time meets its deadline.
```

The local random generators use exactly these ideas. That is why the reference checker is more valuable than fixed samples alone.

11 What to carry forward

Greedy problems become manageable when you separate three things:

1. the local choice;
2. the proof that the local choice is safe;
3. the data structure that implements repeated local choices efficiently.

For easy greedy problems, the proof may be one exchange. For harder Codeforces, AtCoder, and ICPC tasks, the proof may be an invariant that survives many local edits. The habit is the same: never trust the local rule until you can explain why an optimal solution can be made to follow it.

11.1 Reference implementations

The proof, trace, and hostile cases immediately above establish the greedy choice. The missing part of the older source editorial was the concrete implementation. `last_day` is deliberately represented by a separate boolean or `None` state rather than a numeric sentinel because pickup days may be negative. The comparison is strict: `last_day == left` still covers an inclusive interval.

11.2 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        int n;
        cin >> n;
        vector<pair<long long, long long>> intervals(n);
        for (auto &[left, right] : intervals) cin >> left >> right;
        sort(intervals.begin(), intervals.end(), [](const auto &a, const auto &b) {
            if (a.second != b.second) return a.second < b.second;
            return a.first < b.first;
        });
```

```

    long long last_day = LLONG_MIN;
    int answer = 0;
    for (auto [left, right] : intervals) {
        if (last_day < left) {
            ++answer;
            last_day = right;
        }
    }
    cout << answer << '\n';
}
return 0;
}

```

11.3 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    tests = data[index]
    index += 1
    out: list[str] = []
    for _ in range(tests):
        n = data[index]
        index += 1
        intervals = [(data[index + 2 * i], data[index + 2 * i + 1]) for i in range(n)]
        index += 2 * n
        intervals.sort(key=lambda interval: (interval[1], interval[0]))

        last_day: int | None = None
        answer = 0
        for left, right in intervals:
            if last_day is None or last_day < left:
                answer += 1
                last_day = right
        out.append(str(answer))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

11.4 5. Guess the Data Structure: local companion

External Kattis task: <https://open.kattis.com/problems/guessthedatastructure> Local package: [problems/05_guess_data_structure](https://open.kattis.com/problems/05_guess_data_structure)

11.5 Restatement

Count the nonempty contiguous subarrays whose maximum value minus minimum value is at most k . Array values may be negative; only the spread is constrained.

This is the section's blind-practice exercise. It needs two monotonic deques at once: one to expose the current maximum and one to expose the current minimum.

11.6 Observation and trace

For a fixed right endpoint, if $[\text{left}, \text{right}]$ has acceptable spread, every shorter suffix ending at right is also acceptable: removing values cannot increase the maximum or decrease the minimum. Therefore after shrinking to a valid window, all $\text{right} - \text{left} + 1$ starts from left through right contribute.

Keep decreasing values in the maximum deque and increasing values in the minimum deque. Store indices, so expired positions can be removed when left moves.

For 1 3 2 5 and $k=2$:

right	value	valid window after shrinking	new subarrays	answer
1		[1]	1	1
3		[1,3]	2	3
2		[1,3,2]	3	6
5		[5]	1	7

At the last step the spread exceeds two until the left pointer has passed the earlier values.

11.7 Algorithm

```
left = 0
max_deque = empty decreasing deque of indices
min_deque = empty increasing deque of indices
answer = 0

for right from 0 to n-1:
    remove dominated back indices from max_deque
    append right to max_deque
    remove dominated back indices from min_deque
    append right to min_deque
```

```

while a[max_deque.front] - a[min_deque.front] > k:
    remove left from either deque when it is at the front
    left += 1

answer += right-left+1

```

11.8 Correctness proof

Lemma 1. After each update, the front of `max_deque` is an index of the maximum value in the current window, and the front of `min_deque` is an index of its minimum value.

Proof. When a new value arrives, every older maximum-deque back value no larger than it is dominated: the new index lasts longer and is at least as large. Removing those backs preserves all possible future maxima. The minimum argument is symmetric for values no smaller than the new value. Expired front indices are removed when the left endpoint passes them. Thus the surviving fronts are exactly the needed extrema. $\square$

Lemma 2. After the shrinking loop, `[left,right]` has spread at most `k`, and every subarray ending at `right` that starts before `left` is invalid.

Proof. Lemma 1 makes the loop condition exactly the current window's spread. The loop stops only when it is at most `k`. Each discarded left index belongs to a window whose spread was greater than `k`; extending that discarded window farther left cannot reduce its spread, so any earlier start is invalid. $\square$

Theorem. The algorithm returns the required number of subarrays.

Proof. By Lemma 2, exactly the `right-left+1` suffixes ending at each right endpoint are valid. The algorithm adds this number once per endpoint. Every nonempty subarray has one right endpoint, so all and only valid subarrays are counted once. $\square$

11.9 Complexity

Each index enters and leaves each deque at most once. Time is $O(n)$ per test case and memory is $O(n)$ in the worst case. The answer can be $n(n+1)/2$, so use `long long` in C++.

11.10 Implementation traps and tests

- Store indices, not just values, so you can remove expired endpoints.
- Use `<=` for the decreasing maximum deque and `>=` for the increasing minimum deque; retaining only the newest equal value is safe.
- Shrink before adding the count for the current right endpoint.
- Negative values are valid, so do not use a sum-based sliding window.

Test `k=0`, all equal values, alternating extreme values, a one-element array, and the all-equal 200000-element case that forces a 64-bit answer. The random generator uses direct quadratic enumeration on small arrays.

11.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int t;cin>>t;while(t--){int n;long l
```

11.12 Full Python solution

```
from collections import deque
import sys
data = list(map(int, sys.stdin.buffer.read().split()))
index = 1
out = []
for _ in range(data[0]):
    n, k = data[index], data[index + 1]
    index += 2
    values = data[index:index + n]
    index += n
    maximum, minimum = deque(), deque()
    left = 0
    answer = 0
    for right, value in enumerate(values):
        while maximum and values[maximum[-1]] <= value: maximum.pop()
        maximum.append(right)
        while minimum and values[minimum[-1]] >= value: minimum.pop()
        minimum.append(right)
        while values[maximum[0]] - values[minimum[0]] > k:
            if maximum[0] == left: maximum.popleft()
            if minimum[0] == left: minimum.popleft()
            left += 1
        answer += right - left + 1
    out.append(str(answer))
print("\n".join(out))
```

12 Stress testing

For next greater:

```
For each i, scan j=i+1..n-1 and take the first greater value.
```

For sliding maximum:

```
For each window, compute max directly.
```

For shortest subarray:

```
Try every left endpoint and extend right while summing.
```

The local generators use these brute-force checks on small arrays.

13 What to carry forward

The important habit is to identify dominance.

Ask:

- What makes an older candidate useless?
- Should equality remove the older candidate?
- Does the structure need values, indices, or both?
- When does a candidate expire?
- Can each candidate be pushed and popped only once?

These questions are the bridge from simple stacks and queues to later graph BFS, DP optimization, and advanced monotonic data structures.

13.1 6. Assigning Workstations: local companion

External Kattis task: <https://open.kattis.com/problems/workstations> Local package: [problems/06__assigning__workstations](#)

13.2 Restatement and traps

Repeatedly merge two current files, paying their combined size, until one file remains. Minimize total cost. With $n \leq 200000$, trying merge orders is exponential; totals require `long long`. A one-file input costs zero.

13.3 Observation and trace

Always merge the two smallest current sizes. For 4,3,2,6, merge 2+3=5, then 4+5=9, then 6+9=15, totaling 29.

View any merge plan as a binary tree whose leaves are original files. A file size is charged once for every ancestor merge, so smaller sizes should be at least as deep as larger sizes. Thus some optimal tree makes the two smallest files siblings at maximum depth; merge them first and recurse on their sum.

13.4 Algorithm

```

put every size in a min-heap
while heap has at least two values:
    pop a and b
    answer += a+b
    push a+b
print answer

```

13.5 Correctness proof

Lemma. There is an optimal merge plan whose first merge uses two smallest current sizes.

Proof. In an optimal merge tree, choose two sibling leaves at greatest depth. Swapping either of their leaf sizes with a no-larger smallest size can only decrease the weighted depth sum, so the two smallest sizes can occupy those sibling leaves without worsening the plan. Contracting that sibling pair is exactly merging them first. $\square$

Theorem. The heap algorithm has minimum total cost.

Proof. By the lemma, its first merge is compatible with an optimal plan. After replacing the pair by their sum, the remaining problem has the same form. Induction on the number of files proves every greedy merge and therefore the total is optimal. $\square$

13.6 Complexity

$n-1$ heap merges cost $O(n \log n)$ time and $O(n)$ memory.

13.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>>n))return 0;priority

```

13.8 Full Python solution

```

import heapq, sys
def main() -> None:
    data=list(map(int,sys.stdin.buffer.read().split()));q=data[1:];heapq.heapify(q);answer=0
    while len(q)>1:
        total=heapq.heappop(q)+heapq.heappop(q);answer+=total;heapq.heappush(q,total)
    print(answer)
if __name__ == "__main__": main()

```

13.9 Implementation notes and tests

Use a min-heap (`greater<long long>` in C++, `heapq` in Python). Do not sort once: merged sums must re-enter the current frontier. Test one file, equal sizes, a very large size beside small ones, and totals beyond 32-bit range.

13.10 Stress testing

For at most seven files, recursively try every pair of current files and take the least cost; this is independent of the greedy rule. The scale generator uses 200000 ones and a heap-computed expected total to exercise heap work and 64-bit accumulation.

14 What to carry forward

A heap represents the next useful frontier, not an arbitrary bag of values. State exactly what each heap item means, prove that its top is globally next, and check the boundary that removes or advances a frontier item.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

14.1 7. Teque: local companion

External Kattis task: <https://open.kattis.com/problems/teque> Local package: [problems/07_teque](#)

14.2 Restatement and constraint implications

There are n employees with current integer salaries. An operation $! k x$ changes employee k 's salary to x . A query $? a b$ asks how many current salaries lie in the inclusive *value* interval $[a, b]$. There are up to 200,000 employees and operations, and salaries can be negative or as large as 10^9 in magnitude.

The interval is about salary values, not employee indices. Scanning all employees per query is quadratic. A query needs only the number of occurrences of each ordered salary value, which is exactly what a Fenwick tree can store after coordinate compression.

14.3 Coordinate collection and trace

Read and retain all operations first. Collect every initial salary and every new x in an update, sort the distinct values, and give each one a one-based rank. This preserves order: for collected salaries u and v , $u < v$ if and only if $\text{rank}(u) < \text{rank}(v)$.

For initial salaries $[10, 20, 30, 40, 50]$, query $? 15 45$ has four salaries at most 45 and one salary below 15, so its answer is $4 - 1 = 3$. After $! 2 35$, remove one count at rank 20 and add one at rank 35. Then $? 30 35$ returns two. The bounds 15 and 45 need not be coordinates: binary search locates where they fall among actual salary values.

14.4 Algorithm

```
collect initial salaries and every update value; sort and deduplicate them
insert all current salary ranks into a Fenwick frequency tree
```

```
for each operation:
    if ! k x:
        remove the rank of current[k]
        set current[k] = x and add its rank
    else ? a b:
        right = number of coordinate values <= b
        left = number of coordinate values < a
        print BIT.prefix(right) - BIT.prefix(left)
```

Use `upper_bound(coords,b)/bisect_right(coords,b)` for right, and `lower_bound(coords,a)/bisect_left(coords,a)` for left.

14.5 Correctness proof

The Fenwick invariant is: the frequency at each rank equals the number of employees currently earning that corresponding salary. Initialization adds each employee once. An update removes exactly its old salary and adds exactly its new one, so the invariant remains true.

For $[a,b]$, `right` covers exactly salaries at most `b`, while `left` covers exactly salaries below `a`. The two Fenwick prefix sums therefore count those two employee sets under the invariant. Their difference counts exactly the employees with $a \leq \text{salary} \leq b$, so every query answer is correct. QED.

14.6 Complexity, implementation notes, and tests

Sorting coordinates costs $O((n+q) \log(n+q))$. Each update/query uses a constant number of binary searches and Fenwick operations, hence $O(\log(n+q))$; memory is $O(n+q)$. Use `long long` for C++ salary values.

Test an update below every old salary, equal salaries, query endpoints absent from coordinates, and an empty value range. Common mistakes are using a lower bound for the inclusive upper endpoint, or adding the new salary without first removing the old one.

14.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;
    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int index, int delta) {
```

```

        for (int n = (int)bit.size(); index < n; index += index & -index) {
            bit[index] += delta;
        }
    }

    int sum(int index) const {
        int result = 0;
        for (; index > 0; index -= index & -index) result += bit[index];
        return result;
    }
};

struct Operation {
    char type;
    int first;
    long long second;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<long long> salary(n + 1), coords;
    for (int i = 1; i <= n; i++) {
        cin >> salary[i];
        coords.push_back(salary[i]);
    }

    vector<Operation> operations;
    operations.reserve(q);
    for (int i = 0; i < q; i++) {
        char type;
        long long first, second;
        cin >> type >> first >> second;
        operations.push_back({type, (int)first, second});
        if (type == '!') coords.push_back(second);
    }

    sort(coords.begin(), coords.end());
    coords.erase(unique(coords.begin(), coords.end()), coords.end());
    auto rank_of = [&](long long value) {
        return int(lower_bound(coords.begin(), coords.end(), value) - coords.begin()) + 1;
    };

    Fenwick bit((int)coords.size());
    for (int i = 1; i <= n; i++) bit.add(rank_of(salary[i]), 1);

    for (const Operation& operation : operations) {
        if (operation.type == '!') {

```

```

        int employee = operation.first;
        bit.add(rank_of(salary[employee]), -1);
        salary[employee] = operation.second;
        bit.add(rank_of(salary[employee]), 1);
    } else {
        long long low = operation.first;
        long long high = operation.second;
        int before_low = int(lower_bound(coords.begin(), coords.end(), low) - coords.begin());
        int through_high = int(upper_bound(coords.begin(), coords.end(), high) - coords.begin());
        cout << bit.sum(through_high) - bit.sum(before_low) << '\n';
    }
}
return 0;
}

```

14.8 Full Python solution

```

import bisect
import sys

class Fenwick:
    def __init__(self, size: int) -> None:
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index < len(self.bit):
            self.bit[index] += delta
            index += index & -index

    def prefix_sum(self, index: int) -> int:
        total = 0
        while index:
            total += self.bit[index]
            index -= index & -index
        return total

def main() -> None:
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    pointer = 0
    n = int(tokens[pointer])
    q = int(tokens[pointer + 1])
    pointer += 2
    salary = [0] + [int(value) for value in tokens[pointer : pointer + n]]
    pointer += n

```

```

operations: list[tuple[bytes, int, int]] = []
coords = salary[1:]
for _ in range(q):
    kind = tokens[pointer]
    first = int(tokens[pointer + 1])
    second = int(tokens[pointer + 2])
    pointer += 3
    operations.append((kind, first, second))
    if kind == b"!":
        coords.append(second)

coords = sorted(set(coords))
bit = Fenwick(len(coords))

def rank_of(value: int) -> int:
    return bisect.bisect_left(coords, value) + 1

for value in salary[1:]:
    bit.add(rank_of(value), 1)

output: list[str] = []
for kind, first, second in operations:
    if kind == b"!":
        employee, new_salary = first, second
        bit.add(rank_of(salary[employee]), -1)
        salary[employee] = new_salary
        bit.add(rank_of(new_salary), 1)
    else:
        low, high = first, second
        before_low = bisect.bisect_left(coords, low)
        through_high = bisect.bisect_right(coords, high)
        output.append(str(bit.prefix_sum(through_high) - bit.prefix_sum(before_low)))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

15 Testing discussion

The small D cases keep a literal salary list and answer queries by scanning it; they never invoke a reference solution. The scale case combines 100,000 updates with 100,000 range-count queries, so a linear scan cannot pass. The expected outputs are shared by the C++ and Python runners.

16 Detailed upsolve workshop

16.1 A. Hotel Queries

Constraints and trap. n, q are up to about $2e5$. A linear scan for each group is too slow; a plain maximum alone finds feasibility but not the first index. Store the maximum remaining capacity in each segment-tree node.

Trace and pseudocode. With capacities $[3, 1, 4, 1, 5]$, a request of four descends root $\rightarrow$ left/right according to whether the left subtree maximum is at least four, reaches index three, and changes it to zero.

```
if root.max < x: output 0
otherwise descend: go left if left.max >= x, else right
at leaf i: subtract x; recompute max values on the path to root; output i
```

Proof and complexity. Every node is the maximum of exactly its leaves. A feasible left child contains an earlier valid hotel; an infeasible left child contains none, so the descent returns the first valid leaf. Each query changes and rebuilds one root-to-leaf path: $O(\log n)$ time, $O(n)$ memory. Use long long capacities. Test no feasible hotel, the first/last feasible hotel, and repeated requests to one hotel.

16.2 B. List Removals

Constraints and trap. Removing an element from an array/vector shifts all following values and becomes quadratic. Instead, original positions are fixed; a Fenwick tree stores one for alive positions and zero for removed positions.

Trace and pseudocode. For values $[10, 20, 30, 40, 50]$, rank two resolves to original index two, outputs 20, then adds -1. With counts $[1, 0, 1, 1, 1]$, rank three resolves to original index four, outputting 40.

```
initialize BIT with one at every index
for rank k:
    i = first index whose BIT prefix sum >= k
    output value[i]; BIT.add(i, -1)
```

Proof and complexity. The BIT prefix sum is the number of alive original positions at or before its index. Therefore its first prefix reaching k is the k -th current item. Fenwick binary lifting gives $O(\log n)$ per removal and $O(n)$ memory. The search uses one-based indices; test rank one, last rank, and duplicate values. A small mutable-list simulation is the stress oracle.

16.3 C. Mo Distinct Queries

Constraints and trap. Static array, many offline range queries, and no updates signal Mo's algorithm. Recomputing a set for every range is quadratic in the worst case. Coordinate-compress values before using an array frequency.

Trace and pseudocode. For $[1, 2, 1, 3, 2, 4]$, activate $[1, 3]$ to obtain frequencies $1:2, 2:1$ and answer two. Move to $[2, 5]$: remove the first one, add 3 and 2, and obtain three.

```
sort queries by left block, alternating right direction
move [curL, curR] one endpoint at a time
add(pos): if freq[a[pos]] changes 0->1, distinct++
remove(pos): if freq[a[pos]] changes 1->0, distinct--
```

Proof and complexity. The frequency invariant exactly represents the current window, so `distinct` is correct when a query is reached. Block order limits endpoint movement to $O((n+q)\sqrt{n})$ in the standard analysis; memory is $O(n+q)$. Convert endpoints once, use buffered I/O in Python, and test all equal values, all distinct values, singletons, and repeated ranges against `len(set(...))` on small random arrays.

17 What to carry forward

Choose a stored summary that answers the decision you actually need: subtree maximum for first-fit, alive prefix count for dynamic rank, and frequencies for an offline window. The invariant makes both the data structure and its proof mechanical.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

17.1 8. Almost Union-Find: local companion

External Kattis task: <https://open.kattis.com/problems/almostunionfind> Local package: [problems/08_almost_union_find](https://open.kattis.com/problems/08_almost_union_find)

17.2 Restatement

We may deactivate cables in an undirected weighted graph. The remaining network must preserve the original connectivity relation: if two computers were connected by some path before deactivation, they must still have a path afterwards. Different original connected components do not need to become connected.

We want the greatest total cost of deactivated cables. Cable costs are nonnegative, and n , m can both reach 200000, so checking connectivity after each possible removal is far too slow.

17.3 Observation: keep a minimum spanning forest

Let `total` be the sum of all cable costs. Maximizing removed cost is the same as minimizing the cost of the cables we keep:

```
saved cost = total - kept cost
```

Consider one original connected component. The kept cables only need to keep that component connected. A minimum-cost connected subgraph can be reduced to a tree: if it contains a cycle, removing any edge on that cycle keeps it connected and never increases cost because all costs are nonnegative. Thus we need a minimum spanning tree for each original component.

Together, those trees form a **minimum spanning forest**. Kruskal's algorithm already builds exactly that forest if we simply process every edge and never declare a disconnected graph impossible.

17.4 Walk-through

Suppose the graph has two original components:

```
component 1:  1 --4-- 2 --1-- 3
                \      /
                ---3---

component 2:  4 --7-- 5
```

Process edges in increasing cost:

```
2-3, cost 1: keep
1-3, cost 3: keep
1-2, cost 4: skip; it would make a cycle
4-5, cost 7: keep
```

The kept forest costs $1 + 3 + 7 = 11$. All cable costs total 15, so the maximum saving is:

```
15 - 11 = 4
```

An isolated vertex needs no kept cable and contributes neither kept cost nor saved cost.

17.5 Algorithm

```
read all edges and add their weights to total
sort edges by increasing weight
make a DSU with one component per vertex
kept = 0

for every edge (u, v, w) in sorted order:
    if union(u, v) succeeds:
        kept += w

print total - kept
```

Unlike a connected-MST problem, do not check whether exactly $n - 1$ edges were kept. A disconnected input is valid: Kruskal returns one tree for each component.

17.6 Correctness proof

We prove that the algorithm outputs the maximum possible saved cost.

Lemma 1. In every original connected component, there is an optimal set of kept cables that is a spanning tree.

Proof. Any valid kept subgraph is connected within that component. If it contains a cycle, remove an edge from the cycle. Connectivity remains because the other cycle edges give an alternate route. Since cable costs are nonnegative, removing the edge does not increase kept cost. Repeating this removes all cycles and leaves a connected acyclic subgraph, which is a spanning tree. $\square$

Lemma 2. Kruskal’s selected edges have minimum possible total cost among all spanning forests that preserve the original components.

Proof. Restrict attention to one original component. Kruskal processes its edges in the same increasing-cost order as ordinary Kruskal on that component, and only joins different current DSU components. By the cut property, every such selected lightest connecting edge is safe for a minimum spanning tree of that component. Therefore its selected edges form a minimum spanning tree there. Applying this independently to every original component proves that the whole selected forest has minimum total cost. $\square$

Theorem. The algorithm prints the maximum total cost that can be saved.

Proof. By Lemma 1, an optimal valid retained network can be chosen as a spanning forest. By Lemma 2, the algorithm’s forest has no greater kept cost than any such valid network. The input’s total cost is fixed, so subtracting this minimum kept cost from it yields the greatest possible removed cost. Thus the printed value is optimal. $\square$

17.7 Complexity

Sorting dominates the work:

```
Time:     $O(m \log m)$ 
Memory:  $O(n + m)$ 
```

DSU operations are effectively constant amortized time because of path compression and union by size.

17.8 Implementation notes

- Accumulate `total` and `kept` in `long long` in C++; their values can exceed a 32-bit integer.
- Repeated cables are harmless. At most one can be useful for a particular DSU merge; the rest are skipped once their endpoints are connected.
- Zero-cost cables can be selected normally. Keeping or removing an extra zero-cost cycle edge does not change the answer.
- Do not print `IMPOSSIBLE` for a disconnected graph. The requirement is to preserve each original component, not to connect all vertices together.

17.9 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;

    explicit DSU(int n) : parent(n + 1), size(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<tuple<long long, int, int>> edges;
    long long total = 0;
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        edges.emplace_back(w, u, v);
        total += w;
    }

    sort(edges.begin(), edges.end());
    DSU dsu(n);
    long long kept = 0;
    for (auto [w, u, v] : edges) {
        if (dsu.unite(u, v)) {
            kept += w;
        }
    }
}

```

```
    cout << total - kept << '\\n';  
}
```

17.10 Full Python solution

```
import sys  
  
class DSU:  
    def __init__(self, n: int) -> None:  
        self.parent = list(range(n + 1))  
        self.size = [1] * (n + 1)  
  
    def find(self, x: int) -> int:  
        while self.parent[x] != x:  
            self.parent[x] = self.parent[self.parent[x]]  
            x = self.parent[x]  
        return x  
  
    def union(self, a: int, b: int) -> bool:  
        a = self.find(a)  
        b = self.find(b)  
        if a == b:  
            return False  
        if self.size[a] < self.size[b]:  
            a, b = b, a  
        self.parent[b] = a  
        self.size[a] += self.size[b]  
        return True  
  
def main() -> None:  
    data = list(map(int, sys.stdin.buffer.read().split()))  
    n, m = data[:2]  
    edges = []  
    total = 0  
    index = 2  
    for _ in range(m):  
        u, v, w = data[index:index + 3]  
        index += 3  
        edges.append((w, u, v))  
        total += w  
  
    edges.sort()  
    dsu = DSU(n)  
    kept = 0  
    for w, u, v in edges:  
        if dsu.union(u, v):
```

```
        kept += w

    print(total - kept)

if __name__ == "__main__":
    main()
```

17.11 Common wrong answers

- Running a connected-MST solution and printing IMPOSSIBLE for an input with multiple original components.
- Summing only skipped edges without accounting for every cable in `total`.
- Using `int` for the sum of up to 200000 costs of 10^9 .
- Sorting in decreasing order, which keeps an expensive forest and minimizes the saving instead.
- Removing a cable whenever its endpoints are currently connected without considering edge costs; the kept forest must be minimum cost.

17.12 Tests that matter

- a graph with no cables and several isolated vertices, which saves 0;
- one tree, where every cable is necessary and saves 0;
- repeated cables with different costs;
- zero-cost edges and cycles;
- multiple disconnected original components;
- a large graph whose total cost requires 64-bit arithmetic.

18 Stress testing

For tiny graphs, enumerate every subset of cables to keep. For each subset, run ordinary BFS from every vertex before and after removals and keep only subsets with exactly the same reachability relation. The best value of

```
total cost - retained cost
```

is an independent oracle for Kruskal's answer. The local generator uses at most ten edges for this exhaustive check and separately creates a 200000-edge case to exercise sorting and DSU at the intended scale.

19 What to carry forward

Kruskal is useful even when the goal is not phrased as “find an MST.” When a problem asks to preserve connectivity while discarding cost, translate it to:

```
minimum cost needed to keep = minimum spanning forest
maximum removable cost      = total cost - forest cost
```

For disconnected inputs, think component by component rather than forcing a single spanning tree.

19.1 9. Single Source Shortest Path: local companion

External Kattis task: <https://open.kattis.com/problems/shortestpath1> Local package: [problems/09_single_source_shortest_path](#)

19.2 Restatement

Find the least-cost directed route from vertex 1 to vertex n , subject to one extra condition: the route must use an even number of roads. Roads have nonnegative costs, vertices and roads may be revisited, and there can be no valid even-hop route even when an ordinary route exists.

19.3 Constraints and traps

n and m can both be 200000, so enumerating routes or storing a distance for every possible number of hops is impossible. The parity of the hop count, not the exact count, is all that matters. A one-road route to the destination is invalid, while the zero-road route from 1 to itself is valid and costs zero.

Do not keep just one best distance per vertex. A cheaper arrival with odd parity might be unusable, while a more expensive even-parity arrival is the one that can answer the problem.

19.4 Observation and walk-through

For every original vertex u , make two states:

```
(u, 0): u has been reached using an even number of roads
(u, 1): u has been reached using an odd number of roads
```

Every road flips the parity. Thus a directed road $u \rightarrow v$ of cost w becomes two state transitions:

```
(u, 0) -> (v, 1), cost w
(u, 1) -> (v, 0), cost w
```

Start in $(1, 0)$ and seek $(n, 0)$. This is an ordinary nonnegative shortest-path problem on $2n$ states.

In the sample, the tempting route $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ costs 12, but has three roads and therefore ends in state $(5, 1)$. The route $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ costs $3 + 1 + 1 + 1 = 6$ and uses four roads, so it reaches $(5, 0)$ and is valid.

19.5 Algorithm

```
dist[u][parity] = infinity for all vertices and both parities
dist[1][0] = 0
push (0, 1, 0) into a min-heap

while heap is not empty:
    pop (cost, u, parity)
    if cost is stale: continue
    for every road u -> v with weight w:
        next_parity = parity xor 1
        relax dist[v][next_parity] with cost + w

print dist[n][0], or -1 if it stayed infinite
```

19.6 Correctness proof

Lemma 1. Every route from 1 to u using parity p corresponds to a path from state $(1, 0)$ to state (u, p) of the same cost, and vice versa.

Proof. Starting at parity zero, each original road toggles parity exactly once. The expanded graph has precisely the matching parity-toggling transition with the same weight for every original road. Applying this correspondence road by road maps an original route to a state path of equal cost. Reversing the mapping reconstructs an original route from any state path. $\square$

Lemma 2. Dijkstra computes the minimum cost for every reachable parity state.

Proof. By construction every expanded transition has the original road's nonnegative cost. Dijkstra's usual smallest-non-stale-pop invariant therefore applies to the expanded graph, so each finalized state distance is shortest. $\square$

Theorem. The algorithm prints the minimum cost of a valid even-hop route, or -1 exactly when none exists.

Proof. By Lemma 1, valid routes are exactly paths from $(1, 0)$ to $(n, 0)$, with equal costs. Lemma 2 finds the cheapest such state path. If $(n, 0)$ is unreachable, Lemma 1 says no valid route exists, so -1 is correct. $\square$

19.7 Complexity

The expanded graph has $2n$ states and $2m$ transitions.

```
Time:     $O((n + m) \log n)$   
Memory:  $O(n + m)$ 
```

19.8 C++ implementation notes

Use `array<long long, 2>` for the two state distances per vertex. The bitwise expression `parity ^ 1` toggles 0 to 1 and 1 to 0. Use `long long` for all path costs because a shortest route can exceed 32-bit range.

19.9 Python implementation notes

Store distances as `dist[vertex][parity]`, and push `(cost, vertex, parity)` tuples into `heapq`. Python integers do not overflow, but the same stale-entry check is still essential for both correctness and running time.

19.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, long long>>> graph(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        graph[u - 1].push_back({v - 1, w});
    }

    const long long INF = (1LL << 62);
    vector<array<long long, 2>> dist(n, {INF, INF});
    using State = tuple<long long, int, int>;
    priority_queue<State, vector<State>, greater<State>> heap;
    dist[0][0] = 0;
    heap.push({0, 0, 0});

    while (!heap.empty()) {
```

```

    auto [cost, u, parity] = heap.top();
    heap.pop();
    if (cost != dist[u][parity]) continue;
    for (auto [v, weight] : graph[u]) {
        int next_parity = parity ^ 1;
        long long next_cost = cost + weight;
        if (next_cost < dist[v][next_parity]) {
            dist[v][next_parity] = next_cost;
            heap.push({next_cost, v, next_parity});
        }
    }
}

cout << (dist[n - 1][0] == INF ? -1 : dist[n - 1][0]) << '\n';
}

```

19.11 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u, v, weight = data[index:index + 3]
        index += 3
        graph[u - 1].append((v - 1, weight))

    inf = 10**30
    dist = [[inf, inf] for _ in range(n)]
    dist[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        cost, u, parity = heapq.heappop(heap)
        if cost != dist[u][parity]:
            continue
        for v, weight in graph[u]:
            next_parity = parity ^ 1
            next_cost = cost + weight
            if next_cost < dist[v][next_parity]:
                dist[v][next_parity] = next_cost
                heapq.heappush(heap, (next_cost, v, next_parity))

    print(-1 if dist[n - 1][0] == inf else dist[n - 1][0])

```

```
if __name__ == "__main__":  
    main()
```

19.12 Common wrong answers

- Running ordinary Dijkstra with one distance per vertex, which loses parity.
- Printing the cheaper of `dist[n][0]` and `dist[n][1]`; only even parity is valid.
- Starting at (1, 1) instead of (1, 0).
- Counting total edge weight parity instead of the number of roads.
- Marking a state final when it is pushed into the heap rather than skipping stale popped entries.

19.13 Tests that matter

- `n = 1` with no roads, whose answer is zero;
- a one-road path to the target, which is unreachable with even parity;
- an odd-hop cheap route and an even-hop more expensive route;
- zero-cost cycles that change parity without changing the cost;
- parallel roads of different costs;
- a long graph with 200000 roads, to test state count and heap behavior.

20 Common wrong answers

- Using BFS because the graph is “just shortest path”.
- Marking a vertex visited when it is pushed into the heap.
- Forgetting to skip stale heap entries.
- Using 32-bit `int` for distances.
- Applying Dijkstra to negative edges.
- In Discount Route, allowing the discount more than once.

21 Stress testing

For small graphs, compare Dijkstra to Floyd-Warshall. For grids, compare to Floyd-Warshall on the explicit small cell graph or to a slow repeated relaxation. For Discount Route, build the two-layer graph explicitly and run a generic Dijkstra to compare against the optimized state implementation.

For Even-Hop Route, build the $2n$ parity-state graph explicitly on tiny inputs and run Floyd-Warshall over those states. This independent oracle catches the two common modeling errors: starting in odd parity and accepting the odd-parity destination state. A separate long path with one parity-changing shortcut checks the 200000-edge scale.

22 What to carry forward

Dijkstra is the default shortest-path algorithm for nonnegative weighted graphs. The modeling skill is deciding what a state must remember. Once the state graph has nonnegative edge weights, the same invariant and heap template apply.

22.1 10. Get Shorty: local companion

External Kattis task: <https://open.kattis.com/problems/getshorty> Local package: [problems/10_get_shorty](#)

22.2 Restatement

The grid has ordinary open cells, walls, a start, a goal, and lowercase-letter portal cells. Walking to an open four-neighbor cell costs 1. From a letter cell, jumping to any cell with that same letter costs 0. We need the minimum cost from S to G, or -1 when no route exists.

The grid can contain 10^6 cells. It is not enough to use ordinary BFS, because a zero-cost teleport must be processed before a one-cost walk. It is also not enough to scan every matching-letter list every time a portal cell is visited: a grid full of the same letter would become quadratic.

22.3 Model the implicit graph

Use one graph vertex per non-wall grid cell.

```
walk to a four-neighbor open cell    edge cost 1
same-letter teleport                  edge cost 0
```

This is exactly a 0-1 shortest-path problem. Store the positions of each of the 26 letters while reading the grid, then use a deque:

```
cost 0 -> push_front
cost 1 -> push_back
```

22.4 Expanding one portal group once

Suppose the deque first removes a cell labelled **a** at distance **d**. Every other **a** cell is reachable from it for cost 0, so all of them can receive distance **d** immediately. There is no reason to scan the complete **a** list again when another **a** cell is reached later. Mark the letter as expanded after this first scan.

The word *first* matters. In 0-1 BFS, a cell removed from the front has the smallest currently possible distance. Therefore the first removed **a** cell has the minimum distance at which any **a** portal can be reached.

22.5 Walk-through

Consider:

```
S # a # G
. a . a .
. . . . .
```

Walking from **S** to the left **a** costs 2:

```
S -> (1,0) -> left a
```

When that **a** is removed from the deque, expand the **a** group:

```
left a -- cost 0 --> right a
```

The right **a** also has distance 2. Two ordinary moves reach **G**, so the answer is 4. The longer all-walking detour is never preferable.

22.6 Algorithm

```
record positions for each letter while reading the grid
dist[cell] = infinity for every cell
dist[S] = 0
push S into a deque
expanded[letter] = false

while deque is not empty:
    remove cell u from the front

    if u has letter x and expanded[x] is false:
        expanded[x] = true
        for every cell v with letter x:
            relax v using cost 0

    for every open four-neighbor v of u:
```

```
    relax v using cost 1

print dist[G], or -1 if it is still infinity
```

To relax an edge of cost w , update only if:

```
dist[u] + w < dist[v]
```

Then push v to the front for $w = 0$, otherwise to the back.

22.7 Correctness proof

We prove that the algorithm prints the minimum possible cost.

Lemma 1. When a portal letter x is expanded, the current distance of the removed x cell is the minimum distance of any route from S to an x cell.

Proof. 0-1 BFS removes cells from the deque in nondecreasing finalized distance order. The expanded cell is the first removed cell labelled x . If a cheaper route to any x cell existed, that cell would have a smaller distance and would be removed first, a contradiction. $\square$

Lemma 2. Expanding each portal letter at most once preserves every shortest-path distance.

Proof. When x is expanded at distance d , every x cell is relaxed to distance at most d by a zero-cost teleport. By Lemma 1, no x cell can later be reached with a value smaller than d . Thus scanning the same zero-cost teleport edges later cannot improve any distance. $\square$

Lemma 3. Apart from skipped redundant portal scans, the algorithm performs the same successful relaxations as 0-1 BFS on the implicit graph.

Proof. Every walking edge is examined whenever its source cell is removed. For a portal edge, its whole same-letter group is examined at the first and minimum-distance source by Lemma 1; Lemma 2 proves later scans are redundant. Each examined edge is relaxed with its stated cost and sent to the correct end of the deque. $\square$

Theorem. The algorithm prints the minimum travel cost from S to G , or -1 exactly when the goal is unreachable.

Proof. By Lemma 3, the algorithm is 0-1 BFS on all useful implicit graph edges. The 0-1 BFS invariant therefore gives the shortest distance from S to every reachable cell, including G . If G remains at infinity, no sequence of valid walking or teleport edges reached it, so it is unreachable. The printed result is consequently correct. $\square$

22.8 Complexity

Each cell has at most four walking edges. Each portal position appears in one letter list, and that list is scanned once. Therefore:

```
Time:     $O(n * m)$ 
Memory:  $O(n * m)$ 
```

22.9 Implementation notes

- Store letters in 26 lists indexed by `letter - 'a'` or `ord(letter) - ord('a')`.
- Mark a letter expanded before scanning its list.
- A teleport is cost 0, so its improved endpoints go to the deque front.
- A normal grid step costs 1, so it goes to the deque back.
- Do not treat S and G as portal letters; only lowercase characters are portal labels.

22.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<string> grid(n);
    vector<vector<int>> portals(26);
    int start = -1, goal = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            int id = r * m + c;
            char cell = grid[r][c];
            if (cell == 'S') start = id;
            if (cell == 'G') goal = id;
            if ('a' <= cell && cell <= 'z') {
                portals[cell - 'a'].push_back(id);
            }
        }
    }

    const int INF = 1e9;
    vector<int> dist(n * m, INF);
    vector<char> expanded(26, false);
    deque<int> dq;
    dist[start] = 0;
    dq.push_back(start);
    int dr[4] = {1, -1, 0, 0};
    int dc[4] = {0, 0, 1, -1};

    while (!dq.empty()) {
        int id = dq.front();
        dq.pop_front();
        int r = id / m, c = id % m;
```

```

    int current = dist[id];

    char cell = grid[r][c];
    if ('a' <= cell && cell <= 'z' && !expanded[cell - 'a']) {
        expanded[cell - 'a'] = true;
        for (int other : portals[cell - 'a']) {
            if (current < dist[other]) {
                dist[other] = current;
                dq.push_front(other);
            }
        }
    }

    for (int k = 0; k < 4; ++k) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m || grid[nr][nc] == '#') {
            continue;
        }
        int other = nr * m + nc;
        if (current + 1 < dist[other]) {
            dist[other] = current + 1;
            dq.push_back(other);
        }
    }
}

cout << (dist[goal] == INF ? -1 : dist[goal]) << "\\n";
}

```

22.11 Full Python solution

```

import sys
from collections import deque

def main() -> None:
    data = sys.stdin.buffer.read().split()
    n, m = map(int, data[:2])
    grid = [row.decode() for row in data[2:2 + n]]
    portals = [[] for _ in range(26)]
    start = goal = -1

    for r, row in enumerate(grid):
        for c, cell in enumerate(row):
            cell_id = r * m + c
            if cell == "S":
                start = cell_id
            elif cell == "G":

```

```

        goal = cell_id
    elif "a" <= cell <= "z":
        portals[ord(cell) - ord("a")].append(cell_id)

inf = 10**9
dist = [inf] * (n * m)
expanded = [False] * 26
dist[start] = 0
dq = deque([start])

while dq:
    cell_id = dq.popleft()
    r, c = divmod(cell_id, m)
    current = dist[cell_id]
    cell = grid[r][c]

    if "a" <= cell <= "z":
        letter = ord(cell) - ord("a")
        if not expanded[letter]:
            expanded[letter] = True
            for other in portals[letter]:
                if current < dist[other]:
                    dist[other] = current
                    dq.appendleft(other)

    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != "#":
            other = nr * m + nc
            if current + 1 < dist[other]:
                dist[other] = current + 1
                dq.append(other)

print(-1 if dist[goal] == inf else dist[goal])

if __name__ == "__main__":
    main()

```

22.12 Common wrong answers

- Using ordinary BFS, which counts teleports as if they cost one move.
- Putting a zero-cost teleport at the deque back.
- Scanning all cells with letter a every time any a cell is visited.
- Marking a portal letter used when it is first discovered instead of when it is removed at its minimum distance.
- Allowing moves through walls or treating S and G as lowercase labels.

22.13 Tests that matter

- no portals, which reduces to an ordinary shortest grid path;
- a portal shortcut that is cheaper than the fewest-step walk;
- repeated cells of one letter, to catch repeated full-list scans;
- an unreachable goal;
- a portal first reached by a costly route but later reachable more cheaply;
- a large open grid, which checks linear memory and time.

23 Stress testing

On a tiny grid, build the implicit graph literally: from every letter cell, add a zero-cost edge to every equal-letter cell. Then run heap Dijkstra and compare its answer with 0-1 BFS. This is deliberately slower but independent of the portal-group optimization. The local generator uses that oracle on small random grids and also includes a 1000 x 1000 open grid with known answer 1998.

24 What to carry forward

0-1 BFS remains linear only when the transition model is also efficient. A large group of equivalent zero-cost moves should usually be activated once at the first finalized distance:

```
implicit zero-cost group + one-time expansion = linear total work
```

First identify the state and the costs, then account for how often every generated transition is scanned.

24.1 11. Coast Length: local companion

External Kattis task: <https://open.kattis.com/problems/coast> Local package: [problems/11_coast_length](#)

24.2 Restatement

An undirected, unweighted graph contains several station vertices. For every vertex, print the minimum number of edges needed to reach any station. Print -1 when its connected component contains no station.

The graph can have up to 200000 vertices and edges, so running an ordinary BFS separately from every vertex, or separately from every station, can be too slow.

24.3 Key idea: start BFS from every station

Imagine adding one virtual source connected to every station by a zero-length edge. The desired answer is the shortest distance from that virtual source. We do not need to build the virtual source: put every station in the BFS queue initially and assign all of them distance 0.

Then run the usual BFS. When a vertex v is first discovered from u , write

```
dist[v] = dist[u] + 1
```

Vertices in components without stations are never put in the queue, so their initial value -1 remains the correct answer.

24.4 Walk-through

For the chain below, stations are vertices 1 and 5:

```
1 -- 2 -- 3 -- 4 -- 5
```

Initial queue and distances:

```
queue: [1, 5]
dist:  [0, -1, -1, -1, 0]
```

Processing 1 discovers 2; processing 5 discovers 4. Both receive distance 1. Next, either 2 or 4 discovers 3, which receives distance 2. The answer is therefore:

```
0 1 2 1 0
```

It does not matter which same-distance vertex is popped first. BFS always processes all vertices at distance d before any vertex at distance $d + 1$.

24.5 Algorithm

```
build an adjacency list for the undirected graph
set every dist value to -1

for every station s:
    if dist[s] is -1:           # harmless if a station is listed twice
        dist[s] = 0
        push s into the queue

while the queue is not empty:
    u = pop from the front
    for every neighbor v of u:
        if dist[v] is -1:
            dist[v] = dist[u] + 1
            push v into the queue

print dist[1], dist[2], ..., dist[n]
```

24.6 Correctness proof

We prove that the algorithm prints the required distance for every vertex.

Every station is inserted with distance 0, so its printed value is correct. Consider a non-station vertex v that is first discovered from u . BFS processes vertices in nondecreasing distance order. Thus u has the minimum possible distance to any station among vertices that could first reach v . Appending edge (u, v) gives a path of length $\text{dist}[u] + 1$ from a station to v , so the algorithm assigns a reachable distance to v .

Suppose a shorter station-to- v path of length L existed. Let w be the vertex directly before v on that path. Because $L < \text{dist}[v] = \text{dist}[u] + 1$, we have $L - 1 < \text{dist}[u]$. BFS reaches and processes w before u , then discovers v with a value at most L . This contradicts the assignment of the larger value $\text{dist}[u] + 1$. Therefore $\text{dist}[v]$ is the minimum distance to any station.

Finally, an unvisited vertex cannot be connected to a station: a path from a station would make BFS traverse its edges and discover it. Its value remains -1 , which is correct. Hence all printed values are correct.

24.7 Complexity

Each vertex enters the queue at most once and each undirected edge is examined twice. The time complexity is $O(n + m)$ and the adjacency list, distances, and queue use $O(n + m)$ memory.

24.8 Reference implementation in C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    cin >> n >> m >> k;

    vector<vector<int>> graph(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);
    queue<int> q;
    for (int i = 0; i < k; ++i) {
        int station;
        cin >> station;
        if (dist[station] == -1) {
            dist[station] = 0;
            q.push(station);
        }
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```

        for (int v : graph[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    for (int v = 1; v <= n; ++v) {
        cout << dist[v] << (v == n ? "\\n" : " ");
    }
}

```

24.9 Reference implementation in Python

```

import sys
from collections import deque

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, k = data[:3]
    index = 3

    graph = [[] for _ in range(n + 1)]
    for _ in range(m):
        u, v = data[index], data[index + 1]
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    dist = [-1] * (n + 1)
    q = deque()
    for station in data[index:index + k]:
        if dist[station] == -1:
            dist[station] = 0
            q.append(station)

    while q:
        u = q.popleft()
        for v in graph[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)

    print(*dist[1:])

```

```
if __name__ == "__main__":  
    main()
```

24.10 Tests worth checking

- One station in a disconnected component: vertices in other components must remain -1.
- Stations at both ends of a path: the middle vertex uses the closer end.
- Adjacent stations and duplicated station input: every station still has distance 0.
- A long chain with one station at an endpoint: checks both linear time and queue behavior.

24.11 12. All Pairs Shortest Path: local companion

External Kattis task: <https://open.kattis.com/problems/allpairspath> Local package: [problems/12_all_pairs_shortest_path](#)

24.12 Restatement

Every undirected road has a difficulty. A route is judged by its hardest road, not by the sum of its road difficulties. We need the smallest possible value of that maximum on a route from city 1 to city n , or -1 if no route exists. When $n = 1$, the empty route has difficulty 0.

This wording is a trap: it is not an ordinary shortest path. Adding a road of difficulty 4 and one of difficulty 5 gives route difficulty 5, not 9.

24.13 Threshold viewpoint

Fix a candidate limit X . Keep only roads with difficulty at most X .

```
There is a route of difficulty at most X  
if and only if  
1 and n are connected using only roads of difficulty at most X.
```

As X increases, roads only get added, so connectivity can only change from false to true. Sort all roads by difficulty and add them in that order with a DSU. The first difficulty at which cities 1 and n share a component is the answer.

This is related to Kruskal, but we do not need to finish an MST. Stop as soon as the two required cities become connected.

24.14 Walk-through

For these roads:

1-3: 3 1-2: 4 3-4: 5 4-5: 5 2-5: 8

Process increasing difficulties:

```
add 1-3 (3): components of 1 and 5 differ
add 1-2 (4): components of 1 and 5 differ
add 3-4 (5): components of 1 and 5 differ
add 4-5 (5): 1 and 5 are now connected
```

The answer is 5. The connecting route is 1 -> 3 -> 4 -> 5, whose maximum edge difficulty is 5.

24.15 Algorithm

```
if n is 1:
    print 0
    stop

sort all roads by difficulty
create DSU with one component per city

for each road (u, v, w) in sorted order:
    union(u, v)
    if find(1) equals find(n):
        print w
        stop

print -1
```

24.16 Correctness proof

Lemma 1. After all roads of difficulty at most X have been processed, the DSU puts two cities in the same component exactly when they are connected using only roads of difficulty at most X .

Proof. DSU starts with isolated cities and merges exactly the endpoints of every processed road. Thus every DSU merge represents an allowed road path, and every allowed road is included in the merges. Its components are exactly the connected components of the threshold graph. $\square$

Lemma 2. When the algorithm first finds cities 1 and n connected at road difficulty w , there exists a route from 1 to n whose maximum road difficulty is at most w .

Proof. All roads processed so far have difficulty at most w . By Lemma 1, DSU connectivity gives a path using only those roads. $\square$

Lemma 3. No route from 1 to n has maximum road difficulty less than the first reported value w .

Proof. Before processing roads of difficulty w , the two cities were in different DSU components. By Lemma 1, they were disconnected in the graph of all roads with difficulty below w . Such a route therefore cannot exist. $\square$

Theorem. The algorithm prints the required minimum route difficulty, or -1 exactly when city n is unreachable from city 1.

Proof. Lemmas 2 and 3 show that a reported w is feasible and minimal. If the scan ends without connecting the cities, Lemma 1 says no input road path connects them, so -1 is correct. The $n = 1$ case is correct by the empty route convention. $\square$

24.17 Complexity

```
Time:     $O(m \log m)$ 
Memory:  $O(n + m)$ 
```

Sorting roads dominates; DSU operations are effectively constant amortized time.

24.18 Implementation notes

- Use `long long` for road difficulties in C++ even though DSU stores city ids.
- Repeated roads are harmless; DSU ignores a road within an existing component.
- Do not add difficulties along a path.
- Do not return the first road incident to city n ; check DSU connectivity.

24.19 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;
    explicit DSU(int n) : parent(n + 1), size(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
};
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<tuple<long long, int, int>> roads;
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        roads.emplace_back(w, u, v);
    }
    if (n == 1) {
        cout << 0 << "\\n";
        return 0;
    }
    sort(roads.begin(), roads.end());
    DSU dsu(n);
    for (auto [w, u, v] : roads) {
        dsu.unite(u, v);
        if (dsu.find(1) == dsu.find(n)) {
            cout << w << "\\n";
            return 0;
        }
    }
    cout << -1 << "\\n";
}

```

24.20 Full Python solution

```

import sys

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n + 1))
        self.size = [1] * (n + 1)

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        a = self.find(a)
        b = self.find(b)
        if a == b:

```

```

        return
    if self.size[a] < self.size[b]:
        a, b = b, a
    self.parent[b] = a
    self.size[a] += self.size[b]

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    roads = sorted(
        (data[i + 2], data[i], data[i + 1])
        for i in range(2, 2 + 3 * m, 3)
    )
    if n == 1:
        print(0)
        return

    dsu = DSU(n)
    for w, u, v in roads:
        dsu.union(u, v)
        if dsu.find(1) == dsu.find(n):
            print(w)
            return
    print(-1)

if __name__ == "__main__":
    main()

```

24.21 Common wrong answers

- Summing road difficulties and using ordinary shortest-path Dijkstra.
- Building a full MST even after cities 1 and n are connected; correct but unnecessary.
- Declaring -1 merely because a low-difficulty prefix is disconnected.
- Forgetting the one-city empty-route case.

24.22 Tests that matter

- city 1 and city n disconnected;
- one city and no roads;
- a low-sum route with one very difficult road versus a longer route with a lower maximum;
- repeated roads with different difficulties;
- many equal-difficulty roads that connect the endpoints together.

25 Stress testing

For small graphs, use a different formulation: minimax Dijkstra. Let `dist[v]` be the smallest possible maximum edge on a route to `v`, and relax an edge of difficulty `w` with:

```
candidate = max(dist[u], w)
```

Use a heap on that candidate and compare its answer with the DSU threshold sweep. The local generator uses this independent oracle for small random graphs and a direct-answer 200000-edge scale case.

26 What to carry forward

When a route is scored by its worst edge, convert a minimization question into a connectivity threshold:

```
"Can I travel using only edges <= X?"
```

Sorting thresholds and DSU are often simpler than forcing an additive shortest path interpretation.

26.1 13. Arbitrage: local companion

External Kattis task: <https://open.kattis.com/problems/arbitrage> Local package: [problems/13_arbitrage](#)

26.2 Restatement and observation

We need $a^{(b^c) \bmod P}$, where $P = 1000000007$ is prime and a is nonzero modulo P . Fermat's theorem gives:

```
 $a^{(P-1)} = 1 \bmod P$ 
```

so only the exponent modulo $P-1$ matters. Compute the inner exponent with a second binary exponentiation:

```
e = b^c mod (P-1)
answer = a^e mod P
```

For example, $2^{(3^2)} = 2^9 = 512$.

26.3 Constraints and traps

There can be up to 200000 queries, while b and c may be enormous. The outer base is guaranteed not to be divisible by P ; that contract is vital.

```
Never form  $b^c$  as an ordinary integer.
Reducing the exponent modulo  $P-1$  is valid here only because  $\gcd(a, P) = 1$ .
The exponent residue may be zero; pow(a, 0, P) correctly returns 1.
Use 64-bit multiplication in C++: products stay below about  $10^{18}$ .
```

If the base could be zero modulo P , the exponent reduction would need a separate zero/nonzero analysis. This problem deliberately avoids that case.

26.4 Worked trace

Consider $a=2$, $b=3$, $c=5$. First compute the inner exponent modulo $P-1 = 1000000006$ by binary exponentiation:

$$e = 3^5 \bmod (P-1) = 243$$

Fermat permits replacing the true exponent 3^5 with this residue, so the second binary power computes $2^{243} \bmod P$. The two powers use the same tool; only their moduli differ. For a huge c , the first call still examines only the bits of c .

26.5 Algorithm

```
for each query a,b,c:
    exponent = power(b, c, P-1)
    print power(a, exponent, P)
```

Both calls are standard binary exponentiation.

26.6 Correctness proof

Lemma. `power(x,y,m)` returns $x^y \bmod m$.

Proof. Binary exponentiation maintains `result * baseremaining` equal to the original power modulo m ; each bit either multiplies `result` by the current power of two or skips it. When `remaining` becomes zero, the invariant gives the result. $\square$

Theorem. The algorithm prints $a^{(b^c)} \bmod P$.

Proof. The first power call returns $e = b^c \bmod (P-1)$ by the lemma. Because a is nonzero modulo prime P , Fermat implies exponents differing by $P-1$ give the same residue, so $a^{(b^c)} = a^e \bmod P$. The second power call returns that value by the lemma. $\square$

26.7 Complexity

Each query uses two logarithmic powers:

```
Time:  $O(q (\log b + \log c))$ 
Memory:  $O(1)$  besides output
```

26.8 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long power(long long base, long long exponent, long long mod) {
    long long result = 1 % mod;
    for (base %= mod; exponent; exponent >>= 1, base = base * base % mod)
        if (exponent & 1) result = result * base % mod;
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q; cin >> q;
    while (q--) {
        long long a, b, c; cin >> a >> b >> c;
        cout << power(a, power(b, c, MOD - 1), MOD) << '\\n';
    }
}

```

26.9 C++ implementation notes

Use `long long` for the products in binary exponentiation: both operands are less than P , and their product fits in signed 64-bit range. Normalize an input base with `% MOD` if the statement later allows values outside the usual range.

26.10 Full Python solution

```

import sys
MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(data[0]):
        a, b, c = data[1 + 3 * i:4 + 3 * i]
        out.append(str(pow(a, pow(b, c, MOD - 1), MOD)))
    print("\\n".join(out))

if __name__ == "__main__":
    main()

```

26.11 Python implementation notes

Python's three-argument `pow(base, exponent, modulus)` is binary exponentiation implemented in C. It avoids constructing the enormous integer `b ** c`; use it for both nested powers.

26.12 Traps and tests

- Reducing the outer exponent by $P-1$ requires a nonzero modulo P .
- Do not compute b^c as an ordinary integer.
- Test $b=1$, huge c , and base $P-1$ with odd/even exponent.

27 Stress testing

For tiny bases and exponents, compute $b^{**}c$ as an ordinary integer and then multiply a that many times modulo P . This independent slow oracle validates the two modular power calls. The generator also emits 200000 queries with known answer 2 to exercise input and logarithmic query processing.

28 What to carry forward

Under a prime modulus, a nonzero base has exponent period dividing $P-1$. Nested powers are handled from inside out, reducing each exponent at the modulus required by the next power.

14. Knapsack: local companion

External Kattis task: <<https://open.kattis.com/problems/knapsack>>

Local package: [problems/14_knapsack](problems/14_knapsack/README.md)

Restatement and reduction

Let the total of all values be T . If one group receives a subset with sum s , the other receives $T - s$, so the split difference is:

```text

$$|s - (T - s)| = |T - 2s|$$

Thus we need a reachable subset sum as close as possible to  $T / 2$ . The total is at most 100000, so use the section's 0/1 subset-sum DP rather than testing all  $2^n$  partitions.

For 1, 6, 11, 5,  $T = 23$ . Choosing subset {11} gives sums 11 and 12, so the answer is 1.

### 28.1 Constraints and traps

All values are positive integers and their total  $T$  is at most 100000. That total, rather than the number of input values alone, is the capacity dimension: an  $O(nT)$  boolean table is practical, while enumerating  $2^n$  assignments is not.

Every occurrence is a separate item, including equal values.

Each occurrence may be assigned to one group exactly once.

There is no requirement that the two sums are equal.

Only a sum  $s \leq T/2$  needs final inspection: its complementary sum is  $T-s$ .

Scanning above  $T/2$  is redundant. If  $s > T/2$ , the other group has sum  $T-s < T/2$  and represents the same unordered split with the same difference.

## 28.2 Reachability trace

For values 1, 6, 11, 5, begin with only sum zero reachable:

```
after no value: {0}
after 1: {0, 1}
after 6: {0, 1, 6, 7}
after 11: {0, 1, 6, 7, 11, 12, 17, 18}
after 5: ... including 11
```

The target half is  $23/2 = 11.5$ , so the best reachable sum at or below it is 11. The resulting difference is  $23 - 2*11 = 1$ .

When processing 6, update the boolean array from high sums down to low sums. For instance, `possible[7]` may use the old `possible[1]`, but that old value must not already include the current 6. An upward loop would reach `possible[12]` through the newly set `possible[6]`, incorrectly using 6 twice.

## 28.3 Algorithm

```
possible[0] = true
for each value x:
 for s from T down to x:
 possible[s] |= possible[s-x]

answer = minimum T - 2*s over reachable s with s <= T/2
```

The loop descends so `possible[s-x]` is still the state before the current value; each value is used at most once.

## 28.4 Correctness proof

**Lemma 1.** After any prefix of values, `possible[s]` is true exactly when a subset of that prefix sums to `s`.

**Proof.** Initially only the empty subset makes 0. For new value `x`, a subset making `s` either omits `x`, or uses it with an earlier subset making `s-x`. Descending updates implement exactly those two cases without reusing `x`.  $\square$

**Lemma 2.** Every split is represented by a reachable sum `s`  $\leq T/2$ , and its difference is  $T - 2s$ .

**Proof.** Choose the smaller of the two group sums as `s`. The other is  $T-s$ , so its difference is  $(T-s)-s$ .  $\square$

**Theorem.** The algorithm prints the minimum possible difference.

**Proof.** Lemma 1 gives exactly every available group sum. Lemma 2 shows the final scan evaluates every possible split's difference, so its minimum is optimal.  $\square$

## 28.5 Complexity

Time:  $O(n * T)$

Memory:  $O(T)$

where  $T$  is the total input sum.

## 28.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);
 int n;
 cin >> n;
 vector<int> a(n);
 int total = 0;
 for (int& x : a) {
 cin >> x;
 total += x;
 }
 vector<char> possible(total + 1);
 possible[0] = true;
 for (int x : a) {
 for (int s = total; s >= x; --s) {
 possible[s] = possible[s] || possible[s - x];
 }
 }
 int answer = total;
 for (int s = 0; s <= total / 2; ++s) {
 if (possible[s]) answer = min(answer, total - 2 * s);
 }
 cout << answer << "\\n";
}
```

## 28.7 C++ implementation notes

`vector<char>` is a compact boolean table and avoids the unusual proxy behavior of `vector<bool>`. The table has indices 0 through `total`, inclusive. The descending loop condition is `s >= x`, so `s-x` is always a valid index.

## 28.8 Full Python solution

```
import sys

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 values = data[1:]
 total = sum(values)
 possible = bytearray(total + 1)
 possible[0] = 1
 for value in values:
 for current in range(total, value - 1, -1):
 if possible[current - value]:
 possible[current] = 1
 print(min(total - 2 * s for s in range(total // 2 + 1) if possible[s]))

if __name__ == "__main__":
 main()
```

## 28.9 Python implementation notes

Use `bytearray(total + 1)` for a compact mutable reachability table; `possible[0]` must start true. In Python, `range(total, value - 1, -1)` is the descending 0/1 update. Only sums through  $T/2$  need inspection after the DP, although updating through  $T$  keeps the transition simple.

## 28.10 Common wrong answers

- An upward loop accidentally permits a value multiple times.
- Requiring an exactly half-total subset instead of accepting the closest one.
- Deduplicating equal input values even though they are separate items.

## 28.11 Tests that matter

- one value;
- an odd total;
- an exactly balanced input;
- repeated values;
- total sum 100000.

## 29 Stress testing

For at most 18 values, enumerate every bitmask and minimize  $\text{abs}(T - 2*s)$ . This independent brute-force oracle checks the DP on random small cases. The local generator separately uses one hundred values of 1000 for the 100000-sum scale case.

## 30 What to carry forward

Partition questions often become subset-sum DP after expressing both group sums in terms of one selected subset. Derive the algebra first, then choose the capacity state and its loop direction.

### 30.1 15. Clock Pictures: local companion

External Kattis task: <https://open.kattis.com/problems/clockpictures> Local package: [problems/15\\_clock\\_pictures](#)

### 30.2 Restatement and traps

For every prefix length  $L$ , count all substring occurrences of that prefix, including overlaps. With  $n \leq 200000$ , comparing every prefix at every start is quadratic. The whole string itself occurs once, and a singleton prefix can occur many times.

### 30.3 Observation and trace

Let  $pi[i]$  be the longest proper prefix ending at position  $i$ . Every prefix ending at  $i$  contributes one occurrence to  $length\ pi[i]$ ; that occurrence also belongs to every border of that prefix. Count direct  $pi$  values, then propagate counts from long lengths to their failure link:

```
for x in pi: count[x]++
for length from n-1 down to 1:
 count[pi[length-1]] += count[length]
answer[length] = count[length] + 1
```

The final  $+1$  counts the prefix occurrence beginning at position zero. Here is the complete small trace for `ababa`:

|            |   |   |   |   |   |
|------------|---|---|---|---|---|
| index i:   | 0 | 1 | 2 | 3 | 4 |
| character: | a | b | a | b | a |
| pi[i]:     | 0 | 0 | 1 | 2 | 3 |

Directly incrementing the nonzero  $pi$  values gives:

|               |   |   |   |   |   |
|---------------|---|---|---|---|---|
| length L:     | 1 | 2 | 3 | 4 | 5 |
| direct count: | 1 | 1 | 1 | 0 | 0 |

Now propagate in decreasing length. Length 3 passes its one occurrence to its failure link  $\text{pi}[2]=1$ ; the resulting relevant counts are:

```
after L=3: count[1]=2, count[2]=1, count[3]=1
```

Lengths 2 and 1 only pass counts to the zero-length sentinel. Adding the initial occurrence to every positive length gives 3 2 2 1 1. In particular, the occurrence of **aba** also contributes to its border **a**; that is exactly what the failure-link propagation preserves.

## 30.4 Correctness proof

**Lemma.** After propagation,  $\text{count}[L]$  is the number of non-initial occurrences of the prefix of length  $L$ .

**Proof.** Every occurrence ending at a position first contributes to its longest proper prefix match. Its failure-link chain lists exactly all shorter prefixes that also end there. Processing links from longer to shorter passes each occurrence to every such border once.  $\square$

**Theorem.**  $\text{count}[L]+1$  is the total occurrence count of prefix length  $L$ .

**Proof.** The lemma counts all occurrences ending after the initial prefix; the unmatched initial occurrence contributes the one added explicitly.  $\square$

## 30.5 Complexity

The prefix function and failure-link propagation are both  $O(n)$  time and use  $O(n)$  memory.

## 30.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;cin>>s;
```

## 30.7 Full Python solution

```
import sys
def main():
 s=sys.stdin.buffer.readline().strip();n=len(s);pi=[0]*n;cnt=[0]*(n+1)
 for i in range(1,n):
 j=pi[i-1]
 while j and s[i]!=s[j]:j=pi[j-1]
 if s[i]==s[j]:j+=1
 pi[i]=j
 for x in pi:cnt[x]+=1
 for i in range(n-1,0,-1):cnt[pi[i-1]]+=cnt[i]
 print(*(cnt[i]+1 for i in range(1,n+1)))
if __name__=='__main__':main()
```

## 30.8 Tests and stress testing

Test `a`, `aaaa`, and a string with no repeated prefix. For tiny strings, the oracle directly compares every prefix against every legal substring start; this is independent of prefix links. The scale case is 200000 copies of `a`, whose answers are `n, n-1, ..., 1`, exposing quadratic matching.

## 30.9 16. String Matching: local companion

External Kattis task: <https://open.kattis.com/problems/stringmatching> Local package: [problems/16\\_string\\_matching](#)

## 30.10 Restatement

For every input word, find the shortest nonempty prefix shared by exactly one input occurrence. Equal words are still separate occurrences, so no prefix can distinguish one copy from another. A word that is a prefix of a longer word may also have no answer.

This is the blind trie exercise for the section: the task asks for a property of all prefixes of each word, not merely a count for one supplied prefix.

## 30.11 Constraints and traps

There are up to 200,000 words and the same bound on total characters. Testing every prefix against every word is quadratic. The answer is an input-order answer, so duplicate words need repeated `-1` lines rather than being merged.

Do not mistake “the word ends here” for “the prefix is unique.” For `do` and `dog`, the prefix `do` is a prefix of both input words even though only one word ends at that trie node.

## 30.12 Observations and trace

In a trie, store `pass[node]`: the number of input words whose path visits the node. During insertion, increment it after every character. Then a prefix is unique exactly when the node reached by that prefix has `pass == 1`.

Insert the sample’s `cat`, `car`, `dog`, `do`, `cat`. The node for `ca` has `pass=3` because it serves both copies of `cat` and `car`. The node for `car` has `pass=1`, so `car` needs length three. The node for `do` has `pass=2`, and there is no later node for the shorter word `do`, so its answer is `-1`.

## 30.13 Algorithm

```

make a trie whose nodes store child links and pass counts
for every word:
 walk/create its character path
 increment pass at each reached node

for every word in input order:
 walk its trie path from the root
 print the first position whose node has pass = 1
 print -1 if no such position occurs

```

## 30.14 Correctness proof

**Lemma.** For every trie node representing prefix  $p$ ,  $\text{pass}[\text{node}]$  equals the number of input words beginning with  $p$ .

**Proof.** A word visits exactly the nodes for its nonempty prefixes during insertion. The algorithm increments this node once for that word and never for a word that does not have the prefix. Summing over all input occurrences gives the stated count.  $\square$

**Theorem.** The algorithm prints the required shortest unique-prefix length for every word.

**Proof.** By the lemma, a scanned prefix is unique exactly when its node has  $\text{pass}=1$ . The second traversal checks prefixes in increasing length order, so the first such node is the shortest valid prefix. If none is found, every nonempty prefix is shared by at least two occurrences and  $-1$  is correct.  $\square$

## 30.15 Complexity

Let  $L$  be the total input length.

```

Time: $O(L)$
Memory: $O(L)$

```

## 30.16 C++ implementation notes

An `array<int,26>` gives fixed lowercase transitions. A node index of zero is the root; newly allocated children receive positive indices. Increment only the destination node's `pass` count, because the empty prefix is not allowed.

## 30.17 Python implementation notes

A list of dictionaries keeps the implementation direct. Use byte input for large files if desired; the reference decodes after splitting because input is restricted to lowercase ASCII.

## 30.18 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
struct Node { array<int,26> next{}; int pass = 0; };
int main() {
 ios::sync_with_stdio(false); cin.tie(nullptr);
 int n; if (!(cin >> n)) return 0; vector<string> words(n); vector<Node> trie(1);
 for (string &word : words) { cin >> word; int node = 0; for (char ch : word) { int c=ch-'a';
 for (const string &word : words) { int node=0, answer=-1; for (int i=0;i<(int)word.size()
}
```

## 30.19 Full Python solution

```
import sys

def main() -> None:
 words = sys.stdin.buffer.read().split()
 if not words: return
 n = int(words[0]); words = [word.decode() for word in words[1:n + 1]]
 children = [{}]; passed = [0]
 for word in words:
 node = 0
 for char in word:
 if char not in children[node]:
 children[node][char] = len(children); children.append({}); passed.append(0)
 node = children[node][char]; passed[node] += 1
 output = []
 for word in words:
 node = 0; answer = -1
 for length, char in enumerate(word, 1):
 node = children[node][char]
 if passed[node] == 1: answer = length; break
 output.append(str(answer))
 sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__": main()
```

## 30.20 Common wrong answers

Counting only words ending at a node instead of every word passing through it.  
Deduplicating equal strings before answering.  
Returning a prefix for do when dog is also present.

Using the root count as an empty-prefix answer.  
Scanning all words for every candidate prefix.

## 30.21 Tests that matter

Identical words: every copy must print -1.  
A word that is a prefix of another word.  
Words sharing a long prefix before branching.  
A one-word input: answer is 1 for a nonempty word.  
200,000 identical one-letter words: checks linear memory and output.

## 31 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

## 32 Testing discussion

Brute-force small trie prefixes by `word.startswith(prefix)`, palindrome queries by reverse comparison, and distinct substrings with a set of all slices. Then run:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```

### 32.1 17. Max Flow: local companion

External Kattis task: <https://open.kattis.com/problems/maxflow> Local package: [problems/17\\_max\\_flow](#)

### 32.2 Restatement and reduction target

Each project has a possibly negative profit. A directed rule `a b` means that selecting project `a` requires selecting project `b`. Choose any feasible set of projects, including the empty set, with maximum total profit. There can be up to 200 projects and 1,000 rules, so enumerating all subsets is impossible.

This is a maximum-closure problem. A cut represents which projects are chosen: put a project on the source side exactly when it is selected. We will construct the network so that a finite cut is exactly a feasible choice, and its capacity is the profit lost by that choice.

### 32.3 Network construction and trace

Let  $P$  be the sum of all positive project profits. Build a source `s` and sink `t` around the project vertices:

```
profit w > 0: add s -> project with capacity w
profit w < 0: add project -> t with capacity -w
rule a requires b: add a -> b with capacity INF
```

Choose  $INF = P + 1$ . No optimum cut can afford an  $INF$  edge because cutting every positive source edge costs at most  $P$ . Therefore a finite minimum cut never puts `a` on the source side while `b` is on the sink side: it obeys every requirement.

For profits  $[9, -5, 7, -3]$  with rules `1->2`, `3->4`, and `3->2`, selecting all projects earns  $9-5+7-3=8$ . In the corresponding cut, no positive source edge is cut and the two negative project-to-sink edges cost  $5+3=8$ . Since  $P=16$ ,  $P-cut=8$ . Selecting only project 1 and its prerequisite 2 has profit 4 and cuts source edge for project 3 (cost 7) plus project 2's sink edge (cost 5), again  $16-12=4$ .

## 32.4 Algorithm

```
P = sum of positive profits
INF = P + 1
build the stated flow network
min_cut = maximum_flow(s, t)
print P - min_cut
```

The max-flow/min-cut theorem supplies the minimum cut value. Use `long long` for `P`, `INF`, capacities, and the result.

## 32.5 Correctness proof

Consider any feasible selected set  $S$  and place exactly its projects with the source. No prerequisite edge crosses from source to sink because feasibility ensures every requirement of a selected project is also selected. The cut crosses  $s \rightarrow v$  for every unselected positive project, and  $v \rightarrow t$  for every selected negative project. Its capacity is therefore

```
sum(positive profits of unselected projects)
+ sum(absolute negative profits of selected projects)
= P - profit(S).
```

Conversely, a minimum cut cannot cross an `INF` edge: cutting all source edges has cost  $P < INF$ . Thus its source-side projects satisfy every prerequisite, so they form a feasible set, and the same equality shows their profit is  $P - \text{cut\_capacity}$ . Minimizing cut capacity therefore maximizes feasible profit. By max-flow/min-cut, the computed maximum flow is that minimum capacity, so the algorithm prints the optimal profit. QED.

## 32.6 Complexity, implementation notes, and tests

The network has  $n+2$  vertices and  $n+m$  forward edges. It uses  $O(n+m)$  memory. With arbitrary integral capacities, Dinic has worst-case  $O(V^2 E)$  time; the deliberately moderate local bound ( $n \leq 200, m \leq 1000$ ) makes this implementation appropriate in both supplied languages. For substantially larger arbitrary-capacity maximum-closure networks, use a flow implementation and limits whose performance guarantee has been analyzed for that setting. `INF` must be strictly larger than the total positive profit, not a small magic constant. Test all-negative projects (answer zero), a profitable project whose requirements make it unprofitable, independent profitable projects, and rules forming a cycle.

## 32.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

struct Dinic {
 struct Edge {
 int to, reverse;
 long long capacity;
 };

 vector<vector<Edge>> graph;
 vector<int> level, next_edge;

 explicit Dinic(int n) : graph(n), level(n), next_edge(n) {}

 void add_edge(int from, int to, long long capacity) {
 Edge forward{to, (int)graph[to].size(), capacity};
 Edge backward{from, (int)graph[from].size(), 0};
 graph[from].push_back(forward);
 graph[to].push_back(backward);
 }

 bool build_levels(int source, int sink) {
 fill(level.begin(), level.end(), -1);
 queue<int> queue;
 level[source] = 0;
 queue.push(source);
 while (!queue.empty()) {
 int vertex = queue.front();
 queue.pop();
 for (const Edge& edge : graph[vertex]) {
 if (edge.capacity > 0 && level[edge.to] == -1) {
 level[edge.to] = level[vertex] + 1;
 queue.push(edge.to);
 }
 }
 }
 return level[sink] != -1;
 }

 long long send_flow(int vertex, int sink, long long pushed) {
 if (vertex == sink) return pushed;
 for (int& edge_index = next_edge[vertex];
 edge_index < (int)graph[vertex].size(); edge_index++) {
 Edge& edge = graph[vertex][edge_index];
 if (edge.capacity == 0 || level[edge.to] != level[vertex] + 1) continue;
 long long flow = send_flow(edge.to, sink, min(pushed, edge.capacity));
 if (flow == 0) continue;
 edge.capacity -= flow;
 graph[edge.to][edge.reverse].capacity += flow;
 return flow;
 }
 }
};

```

```

 return 0;
 }

 long long max_flow(int source, int sink) {
 long long total = 0;
 const long long INF = numeric_limits<long long>::max() / 4;
 while (build_levels(source, sink)) {
 fill(next_edge.begin(), next_edge.end(), 0);
 while (long long pushed = send_flow(source, sink, INF)) total += pushed;
 }
 return total;
 }
};

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, m;
 if (!(cin >> n >> m)) return 0;
 vector<long long> profit(n);
 long long positive_total = 0;
 for (long long& value : profit) {
 cin >> value;
 if (value > 0) positive_total += value;
 }

 int source = n;
 int sink = n + 1;
 Dinic flow(n + 2);
 long long inf = positive_total + 1;
 for (int project = 0; project < n; project++) {
 if (profit[project] > 0) flow.add_edge(source, project, profit[project]);
 if (profit[project] < 0) flow.add_edge(project, sink, -profit[project]);
 }
 for (int i = 0; i < m; i++) {
 int project, prerequisite;
 cin >> project >> prerequisite;
 flow.add_edge(project - 1, prerequisite - 1, inf);
 }

 cout << positive_total - flow.max_flow(source, sink) << '\n';
 return 0;
}

```

## 32.8 Full Python solution

```

import sys
from collections import deque

```

```

class Dinic:
 def __init__(self, size: int) -> None:
 self.graph: list[list[list[int]]] = [[] for _ in range(size)]
 self.level = [-1] * size
 self.next_edge = [0] * size

 def add_edge(self, start: int, end: int, capacity: int) -> None:
 self.graph[start].append([end, len(self.graph[end]), capacity])
 self.graph[end].append([start, len(self.graph[start]) - 1, 0])

 def build_levels(self, source: int, sink: int) -> bool:
 self.level = [-1] * len(self.graph)
 self.level[source] = 0
 queue = deque([source])
 while queue:
 vertex = queue.popleft()
 for target, _, capacity in self.graph[vertex]:
 if capacity > 0 and self.level[target] == -1:
 self.level[target] = self.level[vertex] + 1
 queue.append(target)
 return self.level[sink] != -1

 def send_flow(self, vertex: int, sink: int, pushed: int) -> int:
 if vertex == sink:
 return pushed
 while self.next_edge[vertex] < len(self.graph[vertex]):
 edge = self.graph[vertex][self.next_edge[vertex]]
 target, reverse, capacity = edge
 if capacity > 0 and self.level[target] == self.level[vertex] + 1:
 flow = self.send_flow(target, sink, min(pushed, capacity))
 if flow:
 edge[2] -= flow
 self.graph[target][reverse][2] += flow
 return flow
 self.next_edge[vertex] += 1
 return 0

 def max_flow(self, source: int, sink: int) -> int:
 total = 0
 while self.build_levels(source, sink):
 self.next_edge = [0] * len(self.graph)
 while pushed := self.send_flow(source, sink, 10**30):
 total += pushed
 return total

def main() -> None:
 tokens = list(map(int, sys.stdin.buffer.read().split()))
 if not tokens:

```

```

 return
 n, m = tokens[0], tokens[1]
 profit = tokens[2 : 2 + n]
 pointer = 2 + n
 positive_total = sum(value for value in profit if value > 0)
 source, sink = n, n + 1
 flow = Dinic(n + 2)
 inf = positive_total + 1
 for project, value in enumerate(profit):
 if value > 0:
 flow.add_edge(source, project, value)
 elif value < 0:
 flow.add_edge(project, sink, -value)
 for _ in range(m):
 project, prerequisite = tokens[pointer] - 1, tokens[pointer + 1] - 1
 pointer += 2
 flow.add_edge(project, prerequisite, inf)
 print(positive_total - flow.max_flow(source, sink))

if __name__ == "__main__":
 sys.setrecursionlimit(300_000)
 main()

```

## 33 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

## 34 Testing discussion

For tiny random networks, compare with a slow augmenting-path implementation; for matching, compare with brute force over small left vertices. Verify cut output by checking every residual-reachable edge and summing original capacities leaving the reached set. For D, enumerate small project subsets and reject ones that violate a prerequisite before comparing their total profit. Run:

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
```

### 34.1 18. Min Cut: local companion

External Kattis task: <https://open.kattis.com/problems/mincut> Local package: [problems/18\\_min\\_cut](#)

### 34.2 Restatement and reduction target

Each project has a possibly negative profit. A directed rule  $a \rightarrow b$  means that selecting project  $a$  requires selecting project  $b$ . Choose any feasible set of projects, including the empty set, with maximum total profit. There can be up to 200 projects and 1,000 rules, so enumerating all subsets is impossible.

This is a maximum-closure problem. A cut represents which projects are chosen: put a project on the source side exactly when it is selected. We will construct the network so that a finite cut is exactly a feasible choice, and its capacity is the profit lost by that choice.

### 34.3 Network construction and trace

Let  $P$  be the sum of all positive project profits. Build a source  $s$  and sink  $t$  around the project vertices:

```
profit w > 0: add s -> project with capacity w
profit w < 0: add project -> t with capacity -w
rule a requires b: add a -> b with capacity INF
```

Choose  $INF = P + 1$ . No optimum cut can afford an  $INF$  edge because cutting every positive source edge costs at most  $P$ . Therefore a finite minimum cut never puts  $a$  on the source side while  $b$  is on the sink side: it obeys every requirement.

For profits  $[9, -5, 7, -3]$  with rules  $1 \rightarrow 2$ ,  $3 \rightarrow 4$ , and  $3 \rightarrow 2$ , selecting all projects earns  $9 - 5 + 7 - 3 = 8$ . In the corresponding cut, no positive source edge is cut and the two negative project-to-sink edges cost  $5 + 3 = 8$ . Since  $P = 16$ ,  $P - \text{cut} = 8$ . Selecting only project 1 and its prerequisite 2 has profit 4 and cuts source edge for project 3 (cost 7) plus project 2's sink edge (cost 5), again  $16 - 12 = 4$ .

## 34.4 Algorithm

```
P = sum of positive profits
INF = P + 1
build the stated flow network
min_cut = maximum_flow(s, t)
print P - min_cut
```

The max-flow/min-cut theorem supplies the minimum cut value. Use `long long` for `P`, `INF`, capacities, and the result.

## 34.5 Correctness proof

Consider any feasible selected set  $S$  and place exactly its projects with the source. No prerequisite edge crosses from source to sink because feasibility ensures every requirement of a selected project is also selected. The cut crosses  $s \rightarrow v$  for every unselected positive project, and  $v \rightarrow t$  for every selected negative project. Its capacity is therefore

```
sum(positive profits of unselected projects)
+ sum(absolute negative profits of selected projects)
= P - profit(S).
```

Conversely, a minimum cut cannot cross an `INF` edge: cutting all source edges has cost  $P < INF$ . Thus its source-side projects satisfy every prerequisite, so they form a feasible set, and the same equality shows their profit is  $P - \text{cut\_capacity}$ . Minimizing cut capacity therefore maximizes feasible profit. By max-flow/min-cut, the computed maximum flow is that minimum capacity, so the algorithm prints the optimal profit. QED.

## 34.6 Complexity, implementation notes, and tests

The network has  $n+2$  vertices and  $n+m$  forward edges. It uses  $O(n+m)$  memory. With arbitrary integral capacities, Dinic has worst-case  $O(V^2 E)$  time; the deliberately moderate local bound ( $n \leq 200, m \leq 1000$ ) makes this implementation appropriate in both supplied languages. For substantially larger arbitrary-capacity maximum-closure networks, use a flow implementation and limits whose performance guarantee has been analyzed for that setting. `INF` must be strictly larger than the total positive profit, not a small magic constant. Test all-negative projects (answer zero), a profitable project whose requirements make it unprofitable, independent profitable projects, and rules forming a cycle.

## 34.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

struct Dinic {
 struct Edge {
 int to, reverse;
 long long capacity;
 };

 vector<vector<Edge>> graph;
 vector<int> level, next_edge;

 explicit Dinic(int n) : graph(n), level(n), next_edge(n) {}

 void add_edge(int from, int to, long long capacity) {
 Edge forward{to, (int)graph[to].size(), capacity};
 Edge backward{from, (int)graph[from].size(), 0};
 graph[from].push_back(forward);
 graph[to].push_back(backward);
 }

 bool build_levels(int source, int sink) {
 fill(level.begin(), level.end(), -1);
 queue<int> queue;
 level[source] = 0;
 queue.push(source);
 while (!queue.empty()) {
 int vertex = queue.front();
 queue.pop();
 for (const Edge& edge : graph[vertex]) {
 if (edge.capacity > 0 && level[edge.to] == -1) {
 level[edge.to] = level[vertex] + 1;
 queue.push(edge.to);
 }
 }
 }
 return level[sink] != -1;
 }

 long long send_flow(int vertex, int sink, long long pushed) {
 if (vertex == sink) return pushed;
 for (int& edge_index = next_edge[vertex];
 edge_index < (int)graph[vertex].size(); edge_index++) {
 Edge& edge = graph[vertex][edge_index];
 if (edge.capacity == 0 || level[edge.to] != level[vertex] + 1) continue;
 long long flow = send_flow(edge.to, sink, min(pushed, edge.capacity));
 if (flow == 0) continue;
 edge.capacity -= flow;
 graph[edge.to][edge.reverse].capacity += flow;
 return flow;
 }
 }
};

```

```

 return 0;
 }

 long long max_flow(int source, int sink) {
 long long total = 0;
 const long long INF = numeric_limits<long long>::max() / 4;
 while (build_levels(source, sink)) {
 fill(next_edge.begin(), next_edge.end(), 0);
 while (long long pushed = send_flow(source, sink, INF)) total += pushed;
 }
 return total;
 }
};

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, m;
 if (!(cin >> n >> m)) return 0;
 vector<long long> profit(n);
 long long positive_total = 0;
 for (long long& value : profit) {
 cin >> value;
 if (value > 0) positive_total += value;
 }

 int source = n;
 int sink = n + 1;
 Dinic flow(n + 2);
 long long inf = positive_total + 1;
 for (int project = 0; project < n; project++) {
 if (profit[project] > 0) flow.add_edge(source, project, profit[project]);
 if (profit[project] < 0) flow.add_edge(project, sink, -profit[project]);
 }
 for (int i = 0; i < m; i++) {
 int project, prerequisite;
 cin >> project >> prerequisite;
 flow.add_edge(project - 1, prerequisite - 1, inf);
 }

 cout << positive_total - flow.max_flow(source, sink) << '\n';
 return 0;
}

```

## 34.8 Full Python solution

```

import sys
from collections import deque

```

```

class Dinic:
 def __init__(self, size: int) -> None:
 self.graph: list[list[list[int]]] = [[] for _ in range(size)]
 self.level = [-1] * size
 self.next_edge = [0] * size

 def add_edge(self, start: int, end: int, capacity: int) -> None:
 self.graph[start].append([end, len(self.graph[end]), capacity])
 self.graph[end].append([start, len(self.graph[start]) - 1, 0])

 def build_levels(self, source: int, sink: int) -> bool:
 self.level = [-1] * len(self.graph)
 self.level[source] = 0
 queue = deque([source])
 while queue:
 vertex = queue.popleft()
 for target, _, capacity in self.graph[vertex]:
 if capacity > 0 and self.level[target] == -1:
 self.level[target] = self.level[vertex] + 1
 queue.append(target)
 return self.level[sink] != -1

 def send_flow(self, vertex: int, sink: int, pushed: int) -> int:
 if vertex == sink:
 return pushed
 while self.next_edge[vertex] < len(self.graph[vertex]):
 edge = self.graph[vertex][self.next_edge[vertex]]
 target, reverse, capacity = edge
 if capacity > 0 and self.level[target] == self.level[vertex] + 1:
 flow = self.send_flow(target, sink, min(pushed, capacity))
 if flow:
 edge[2] -= flow
 self.graph[target][reverse][2] += flow
 return flow
 self.next_edge[vertex] += 1
 return 0

 def max_flow(self, source: int, sink: int) -> int:
 total = 0
 while self.build_levels(source, sink):
 self.next_edge = [0] * len(self.graph)
 while pushed := self.send_flow(source, sink, 10**30):
 total += pushed
 return total

def main() -> None:
 tokens = list(map(int, sys.stdin.buffer.read().split()))
 if not tokens:

```

```

 return
 n, m = tokens[0], tokens[1]
 profit = tokens[2 : 2 + n]
 pointer = 2 + n
 positive_total = sum(value for value in profit if value > 0)
 source, sink = n, n + 1
 flow = Dinic(n + 2)
 inf = positive_total + 1
 for project, value in enumerate(profit):
 if value > 0:
 flow.add_edge(source, project, value)
 elif value < 0:
 flow.add_edge(project, sink, -value)
 for _ in range(m):
 project, prerequisite = tokens[pointer] - 1, tokens[pointer + 1] - 1
 pointer += 2
 flow.add_edge(project, prerequisite, inf)
 print(positive_total - flow.max_flow(source, sink))

if __name__ == "__main__":
 sys.setrecursionlimit(300_000)
 main()

```

## 35 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

## 36 Testing discussion

For tiny random networks, compare with a slow augmenting-path implementation; for matching, compare with brute force over small left vertices. Verify cut output by checking every residual-reachable edge and summing original capacities leaving the reached set. For D, enumerate small project subsets and reject ones that violate a prerequisite before comparing their total profit. Run:

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
```

### 36.1 19. Distributing Ballot Boxes: local companion

External Kattis task: <https://open.kattis.com/problems/distributingballotboxes> Local package: [problems/19\\_distributing\\_ballot\\_boxes](https://open.kattis.com/problems/19_distributing_ballot_boxes)

### 36.2 Restatement and predicate

Machines run in parallel; machine  $i$  produces  $\text{floor}(\text{minutes} / \text{time}[i])$  items by a proposed deadline. The question “can we make at least goal items by  $x$ ?” is monotone: if it is true at  $x$ , it stays true at every later minute.

### 36.3 Bounds, trace, and algorithm

Zero minutes is infeasible because the goal is positive.  $\text{min}(\text{time}) * \text{goal}$  is feasible because the fastest machine can work alone; the local bounds keep it inside signed 64-bit range. With times 2 and 3, seven items need nine minutes: at eight minutes production is  $4+2=6$ , while at nine it is  $4+3=7$ .

```
lo = 0; hi = fastest_time * goal
while lo < hi:
 mid = lo + (hi-lo)/2
 if sum(mid / time[i]) >= goal: hi = mid
 else: lo = mid + 1
print lo
```

## 36.4 Correctness proof

For any  $x$ , each machine's production is exactly  $\text{floor}(x/\text{time}[i])$ , so the predicate computes total production correctly. Increasing  $x$  cannot decrease any term, hence feasibility is monotone. The binary-search invariant keeps  $lo$  infeasible (unless it is the answer boundary) and  $hi$  feasible; each iteration retains the first feasible value in the interval. When they meet, that value is the minimum feasible deadline, which is the requested answer.

## 36.5 Complexity and implementation notes

Each predicate is  $O(n)$ , so the solution is  $O(n \log(\text{min\_time} * \text{goal}))$  time and  $O(n)$  memory for the input. In C++, use `long long` and stop summing once total reaches goal to avoid unnecessary growth. Python integers are safe, but the same early break is useful. Test one machine, goal one, equal machine times, and a case near the 64-bit bound. The random tests use minute-by-minute brute force only on small instances plus a 200000-machine scale case.

## 36.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int test_count;
 cin >> test_count;

 while (test_count--) {
 int n;
 long long goal;
 cin >> n >> goal;

 vector<long long> time(n);
 for (long long& x : time) {
 cin >> x;
 }

 long long lo = 0;
 long long hi = *min_element(time.begin(), time.end()) * goal;

 while (lo < hi) {
 long long mid = lo + (hi - lo) / 2;
 long long made = 0;

 for (long long machine_time : time) {
 made += mid / machine_time;
 }
 }
 }
}
```

```

 if (made >= goal) break;
 }

 if (made >= goal) {
 hi = mid;
 } else {
 lo = mid + 1;
 }
}

cout << lo << '\n';
}
}

```

## 36.7 Full Python solution

```

import sys

def minimum_deadline(time: list[int], goal: int) -> int:
 lo = 0
 hi = min(time) * goal

 while lo < hi:
 mid = (lo + hi) // 2
 made = 0

 for machine_time in time:
 made += mid // machine_time
 if made >= goal:
 break

 if made >= goal:
 hi = mid
 else:
 lo = mid + 1

 return lo

data = list(map(int, sys.stdin.buffer.read().split()))
i = 1
out = []

for _ in range(data[0]):
 n = data[i]
 goal = data[i + 1]
 i += 2

```

```

time = data[i:i + n]
i += n
out.append(str(minimum_deadline(time, goal)))

sys.stdout.write("\n".join(out))

```

## 36.8 20. Catering: local companion

External Kattis task: <https://open.kattis.com/problems/catering> Local package: [problems/20\\_catering](#)

## 36.9 State expansion

The free-edge coupon is part of the route history. Reaching the same vertex before and after using it are different situations, so use states `(vertex,used)` where `used` is zero or one. From `(v,used)`, an edge of weight `w` gives `(to,used)` at cost `w`; when `used=0`, it also gives `(to,1)` at cost zero.

All expanded-edge costs are nonnegative, so run Dijkstra from `(1,0)`. The answer is the smaller distance to `(n,0)` and `(n,1)`.

## 36.10 Correctness proof

Every legal original route maps to exactly one expanded path by marking whether its coupon has already been used. Its expanded cost equals its paid cost. Conversely, every expanded path uses at most one zero-cost coupon transition, so it maps to a legal route of the same cost. Dijkstra returns shortest paths in this nonnegative expanded graph; therefore the reported minimum is optimal.

## 36.11 Complexity and tests

The expanded graph has  $2n$  states and at most  $4m$  directed transitions, so Dijkstra costs  $O((n+m) \log n)$  time and  $O(n+m)$  memory. Test one edge (answer zero), a route where the most expensive edge is not on the ordinary shortest path, and a long chain. The tiny oracle uses repeated edge relaxation on both coupon layers; the scale case is a 200000-vertex chain.

## 36.12 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin

```

## 36.13 Full Python solution

```
import sys,heapq
def main():
 d=list(map(int,sys.stdin.buffer.read().split()))
 if not d:return
 n,m=d[:2];g=[[] for _ in range(n)];i=2
 for _ in range(m):
 a,b,w=d[i]-1,d[i+1]-1,d[i+2];i+=3;g[a].append((b,w));g[b].append((a,w))
 inf=10**30;dist=[[inf]*n for _ in range(2)];dist[0][0]=0;pq=[(0,0,0)]
 while pq:
 cost,used,v=heapq.heappop(pq)
 if cost!=dist[used][v]:continue
 for to,w in g[v]:
 if cost+w<dist[used][to]:dist[used][to]=cost+w;heapq.heappush(pq,(cost+w,used,to))
 if not used and cost<dist[1][to]:dist[1][to]=cost;heapq.heappush(pq,(cost,1,to))
 print(min(dist[0][-1],dist[1][-1]))
if __name__=='__main__':main()
```

## **37 What to carry forward**